Git Proxy — Master Specification & Single Source of Truth (SSoT)

> **Codename:** `gitpx` (Git Proxy eXtended) **Document Class:** Development-Ready Engineering Specification **Status:** `APPROVED — SSoT` (supersedes `Git_Proxy_Idea.md`, `Git_Proxy_Idea_Fixed.md`, `Git_Proxy_Idea_Fixed_2.md`) **Version:** `4.0.0 — Unified Blueprint` **Scope:** End-to-end blueprint — from nano-level implementation details to planetary-scale architecture — for a zero-cost, transparent, AI-augmented, cryptographically-verifiable **Git Fan-Out Proxy** that serves as a single authoritative upstream while transparently replicating every Git operation to an arbitrary, hot-reloadable set of remote Git services (GitHub, GitLab, Gitee, GitFlic, GitVerse, Bitbucket, Forgejo, Codeberg, and any generic Git HTTP/SSH endpoint). **Audience:** Engineering leadership, project managers, platform engineers, SREs, security engineers, frontend/mobile developers, ML/AI engineers, QA engineers, tech writers. **Contract:** Every statement in this document is actionable. Every task maps to a concrete deliverable. Every risk has an owner and a mitigation. Nothing is aspirational unless explicitly marked `[FUTURE]`.

---

0. How to Read This Document

This document is the **sole authoritative reference** for the `gitpx` project. It consolidates all prior research (`Git_Proxy_Idea.md`), first-pass fact-check (`Git_Proxy_Idea_Fixed.md`), and second-pass audit (`Git_Proxy_Idea_Fixed_2.md`), resolves every inconsistency found between them, and extends the blueprint with deep innovations across UI/UX, low-level systems, orchestration, AI/LLM usage, computer vision, GPU acceleration, virtualization, modern build systems, scripting SDKs, observability, real-time event dispatch, and client ecosystems (web, desktop, mobile for Linux, Windows, macOS, Android, iOS, HarmonyOS NEXT, Aurora OS).

Document conventions:

- `[FACT-CHECKED]` — Statement verified across sources or authoritative docs.
- `[VERIFY-AT-INTEGRATION]` — Value subject to change by third party (e.g., LLM rate limits); must be re-confirmed when the relevant task begins.
- `[NEW]` — Additions not present in any of the three source documents.
- `[RESOLVED]` — Inconsistency between source documents that this SSoT resolves definitively.
- `R-xxx` — Risk register identifier.
- `T-x.y.z` — Task identifier in the phased breakdown (Phase.SubPhase.Task).

Reading path:

- **Executives / PMs** — Read §1 (Vision), §2 (Approach), §3 (Phasing Summary), §16 (Roadmap), §23 (Scaling).
- **Platform / SRE engineers** — Read §4–§9 (Architecture, Infrastructure, Safety), §13 (Observability), §14 (Testing), §16 (Roadmap).
- **Go/Rust engineers** — Read §5 (Smart-Worker), §6 (Universal Adapter), §7 (Innovations), §15 (Acceleration), §17 (Appendix B code).
- **AI engineers** — Read §10 (AI Integration), §11 (Provider Matrix), §12 (Multi-Agent).
- **Frontend / Mobile engineers** — Read §18 (Clients), §19 (Web), §20 (Desktop), §21 (Mobile), §22 (CLI/TUI/Extensions).
- **QA engineers** — Read §14 (Testing), §13 (Observability), §8 (Risks).
- **Security engineers** — Read §8 (Risk Landscape), §9 (Mitigations), §7 (Provenance), §11.3 (Secret handling).

---

1. Executive Summary & Product Vision

1.1 The Problem

Modern software teams increasingly need **code presence on multiple Git hosting platforms simultaneously**. Reasons include:

- **Geopolitical redundancy** — GitHub may be unreachable from some jurisdictions; Gitee, GitFlic, and GitVerse serve mainland China and Russian markets respectively.
- **Vendor lock-in mitigation** — A single-platform outage, policy change, or account suspension can vaporize years of work.
- **Audience reach** — Open-source maintainers want discoverability on every major forge.
- **Compliance & sovereignty** — Certain clients/governments mandate on-shore mirroring.
- **Supply-chain resilience** — Mirrors function as live, warm, drop-in replacements.

Existing tooling forces the developer to be aware of each destination — either through multi-URL remote configuration, specialized CLI wrappers (`mgit`, `git-multi-sync`), or manual scheduled mirroring (which is not real-time and loses commits on failure).

1.2 The Vision

Build a **Transparent Git Fan-Out Proxy** that is:

1. **Indistinguishable from a normal Git service** — developers `git clone`, `git push`, `git fetch` exactly as they would against GitHub. No new commands. No new workflows. No plugins. No education required.
2. **Authoritative as the source of truth** — the proxy holds the canonical, immutable, cryptographically-signed copy of every commit, branch, and tag.
3. **Silently multi-destination** — every accepted push is replicated to an arbitrary number of configured upstreams in parallel, with bulletproof per-remote queuing, retry, rate-limit awareness, and cryptographic attestation.
4. **Hot-reconfigurable** — adding, disabling, or removing an upstream is a single-line config change that takes effect within seconds, without service restart, without risking the codebase.
5. **Zero-cost-by-default** — the entire production system runs on Always-Free cloud tiers and open-source components. Total cost of ownership = $0.
6. **Micro-to-Planetary** — the very same architecture supports a team of one on a Raspberry Pi and a global enterprise on Kubernetes with no code rewrites.
7. **AI-augmented** — LLM agents perform code review, security scanning, log triage, and incident response using free/cheap APIs with local fallbacks.
8. **Verifiably safe** — every push is signed via Sigstore/Cosign and recorded in the Rekor transparency log. Mathematical proof of what was deployed where and when.

1.3 Non-Negotiable Mandates

#	Mandate	Measurable Definition of Done
M-1	**Absolute integrity**	Zero data loss across a 1-year synthetic chaos-test workload of 10^6 pushes.
M-2	**Zero friction UX**	A new developer completes first push in ≤ 5 minutes using only vanilla `git`.
M-3	**Dynamic configuration**	Adding an upstream takes ≤ 60 s, triggers no service restart, cannot corrupt the repo.
M-4	**Near-zero cost**	Total monthly infra cost ≤ $0.00 for the reference 1–5 user deployment.
M-5	**Scale-on-demand**	Moving from 1 VM to a 10-node K8s cluster requires only config & infra changes — **zero code rewrites**.
M-6	**Provable safety**	Every commit pushed to every upstream has a verifiable Sigstore signature in Rekor.
M-7	**No ports exposed**	The host VM has zero public ingress ports (Cloudflare Tunnel egress-only).
M-8	**Polyglot clients**	Supported clients: Web, Linux/Win/macOS desktop, Android, iOS, HarmonyOS NEXT, Aurora OS, CLI, TUI, browser extension, IDE extensions.

---

2. Evolution of the Approach & Technology Evaluation

2.1 Candidates Considered and Rejected

Multiple open-source solutions were evaluated before committing to the chosen architecture. The table below consolidates the analyses from all source documents.

Candidate	Primary Purpose	Verdict	Reason for Rejection
`git-cdn`, `goblet`	Caching proxy for reads (clone/fetch)	❌	Read-only; no push handling.
`refractr`, `git-mirrorer`	Scheduled mirroring	❌	Not real-time; can drop commits between schedules.
`mgit`, `git-multi-sync-tool`	Client-side multi-push	❌	Violates M-2 (developer must run a custom command).
`josh`	Monorepo virtualization / history filtering	❌	Different problem domain.
`git-server-proxy`, `finos/git-proxy`	Policy gatekeeper proxy	❌	Gatekeeper, not fan-out.
Custom `post-receive` bash hook	Bare repo + Nginx/`fcgiwrap` + shell fan-out	⚠️	Works for v0 demos; lacks dashboard, user/org mgmt, robust failure handling. Useful as **fallback** and **reference implementation** only.

Candidate	Primary Purpose	Verdict	Reason for Rejection
Local multi-URL push (`git remote set-url --add --push`)	Built-in Git multi-remote push	❌	Decentralized; slow; fragile; violates M-2.
Gitea's native push mirror	Gitea built-in feature	⚠️	Known to suffer queue deadlock when one mirror fails (R-005); chosen as presentation layer but **its mirror engine is disabled** in favor of the Smart-Worker.

2.2 Chosen Architecture: Gitea + Event-Driven Custom Smart-Worker

The definitive architecture: **Gitea/Forgejo** serves as the forge (UI, auth, data store), while a **custom event-driven Go microservice** (the **Smart-Worker**) owns all fan-out logic. Native Gitea push mirroring is **disabled globally**.

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          Developer laptop                                │
│                    (unaware of any upstream besides "origin")            │
└──────────────────────────────────────────────────────────────────────────┘
                    │ git push origin main (HTTPS/SSH)
                    ▼
          ┌──────────────────────────┐
          │  Cloudflare Tunnel       │  (zero open ingress ports on host)
          └──────────────────────────┘
                    ▼
     ┌──────────────────────────────────────┐
     │   Caddy (TLS) → Gitea + PostgreSQL   │◄── canonical source of truth
     └──────────────────────────────────────┘
            │ Gitea webhook (HMAC-signed JSON)
            ▼
     ┌──────────────────────────┐
     │ Webhook Receiver (Go)    │──► Gitleaks secret scan (blocking gate)
     └──────────────────────────┘
            │ XADD event to per-repo Redis Stream
            ▼
     ┌──────────────────────────────────────┐
     │  Redis Streams (persistent AOF)      │  per-remote consumer groups
     └──────────────────────────────────────┘
            │ XREADGROUP (blocking)
            ▼
     ┌──────────────────────────────────────┐
     │       Smart-Worker (Go)              │
     │   • Universal Git Adapter            │
     │   • Provider-specific rate-limiters  │
     │   • LFS (Dragonfly P2P)              │
     │   • Sigstore/Cosign signing          │
     │   • Prometheus /metrics              │
     └──────────────────────────────────────┘
       ▼         ▼         ▼         ▼
    ┌──────┐  ┌──────┐  ┌──────┐  ┌──────┐
    │GitHub│  │GitLab│  │ Gitee │  │ ...  │  (configurable fan-out)
    └──────┘  └──────┘  └──────┘  └──────┘
```

Why this decomposition:

- **Gitea** provides battle-tested UI, auth (OAuth2, LDAP, OIDC, SAML via Coolify/Authelia), org/team mgmt, branch protection, PRs, issues, a first-class REST & GraphQL API, and CPU-efficient Go implementation (256 MB baseline). `[FACT-CHECKED]`
- **Redis Streams** (not Lists, not Pub/Sub) because they are **durable** (with AOF), support **consumer groups** (horizontal scale), and **XPENDING/XACK** (at-least-once semantics with redelivery). `[FACT-CHECKED]`
- **Go** for the Smart-Worker because `go-git` is mature, cross-compilation is trivial, binary deployment is simple, and concurrency primitives match the fan-out problem perfectly. `[FACT-CHECKED]`
- **Decoupling via Redis** means the worker can crash, be redeployed, or scaled horizontally across VMs — without losing a single push event.

---

3. Zero-Cost Infrastructure & Hosting Strategy

3.1 The Free-Tier Stack

Component	Provider / Service	Spec / Offering	Fit Rationale	`[VERIFY]`
Primary compute	Oracle Cloud Infrastructure (OCI) Always Free	`VM.Standard.A1.Flex` — up to 4 Ampere Arm OCPUs + 24 GB RAM + 200 GB block storage	Sufficient for Gitea + PostgreSQL + Redis + Smart-Worker + LiteLLM + Ollama 7B model.	`[VERIFY-AT-INTEGRATION]`

Component	Provider / Service	Spec / Offering	Fit Rationale	[VERIFY]
Alt compute (EU/US)	Google Cloud Platform Free Tier	`e2-micro` (2 vCPU shared, 1 GB RAM), 30 GB HDD	Backup region, geographic redundancy.	[VERIFY-AT-INTEGRATION]
Alt compute (PaaS)	Zeabur / Railway / Fly.io	~$5/mo starter tier	Zero-maintenance path for non-ops teams.	
Alt compute (on-prem)	Raspberry Pi 5 / mini-PC / NAS	4–8 GB RAM typical	Full data sovereignty, air-gapped option.	
Networking / TLS	Cloudflare Tunnels (`cloudflared`) + Cloudflare DNS	Unlimited zero-trust egress tunnels, free SSL, DDoS protection	Eliminates M-7 (no open ports). Removes need for Let's Encrypt plumbing.	[FACT-CHECKED]
Alt networking	Caddy (bundled with auto-HTTPS via Let's Encrypt)		For on-prem / air-gapped fallback.	
Backup storage	Backblaze B2	10 GB free, S3-compatible, Object Lock	Immutable encrypted backups.	[VERIFY-AT-INTEGRATION]
Alt backup	Oracle Object Storage	20 GB Always Free	Same-provider redundancy option.	[VERIFY-AT-INTEGRATION]
CI / CD	GitHub Actions	2,000 min/mo for private; unlimited for public	Runs E2E, chaos, security pipelines on the proxy from outside the proxy.	[VERIFY-AT-INTEGRATION]
Alt CI	Gitea Actions (self-hosted on the same VM)	Unlimited minutes (uses host CPU)	Full independence from GitHub.	
Container registry	GitHub Container Registry (`ghcr.io`)	Free for public	Hosts Smart-Worker, Webhook Receiver images.	[FACT-CHECKED]
Error tracking	Sentry Developer plan	5,000 errors/mo free	Real-time crash/ANR monitoring for mobile clients.	[VERIFY-AT-INTEGRATION]
Transactional email	Resend / Mailgun free tier / Brevo	~300–3,000/mo free	Alert notifications.	[VERIFY-AT-INTEGRATION]
Push notifications	Firebase Cloud Messaging	Free (with quotas)	Mobile client real-time event delivery.	
Mobile app distribution (Android alt)	IzzyOnDroid, F-Droid	Free	FOSS-friendly distribution without Play Store.	

[NEW] **Total infrastructure cost for reference deployment (1–5 users): $0.00 / month.**

3.2 Deployment Topology

```
                    INTERNET
                       │
                       ▼
              ┌──────────────────────┐
              │  Cloudflare Edge     │
              │  (WAF, DDoS, CDN)    │
              └──────────┬───────────┘
                         │ Cloudflare Tunnel (outbound only)
                         ▼
╔═══════════════════════════════════════════════════════════╗
║           Oracle Cloud A1.Flex VM (ARM)                 ╖
│  ┌──────────────────────────────────────────────────┐   ╖
│  │ Docker / Podman                                  │   ╖
│  │  ┌──────┐ ┌──────────┐ ┌──────────────┐         │   ╖
│  │  │Gitea │ │PostgreSQL│ │Redis (AOF)   │         │   ╖
│  │  └──────┘ └──────────┘ └──────────────┘         │   ╖
│  │  ┌──────────┐ ┌──────────┐                      │   ╖
│  │  │Smart-Worker│ │Webhook Recv│                  │   ╖
│  │  └──────────┘ └──────────┘                      │   ╖
│  │  ┌──────┐ ┌──────┐ ┌──────────┐                │   ╖
│  │  │LiteLLM│ │Ollama│ │Dragonfly LFS│             │   ╖
│  │  └──────┘ └──────┘ └──────────┘                │   ╖
│  │  ┌──────────────────────────────────────────┐   │   ╖
│  │  │ Prometheus / Grafana / Loki /            │   │   ╖
│  │  │ Alertmanager / cAdvisor / NodeExporter   │   │   ╖
```

```
│  │  │                                                  │  │  │
│  │  │                                                  │  │  │
│  │  │                                                  │  │  │
```

```
HOST = db:5432
NAME = gitea
USER = gitea
PASSWD = \${GITEA_DB_PASSWORD}

[repository]
DISABLE_MIRRORS = true                ; CRITICAL: disable native push-mirrors
DEFAULT_PUSH_CREATE_PRIVATE = true
DEFAULT_REPO_UNITS = repo.code,repo.releases,repo.issues,repo.pulls

[webhook]
DELIVER_TIMEOUT = 10
ALLOWED_HOST_LIST = private,127.0.0.1,172.16.0.0/12,10.0.0.0/8

[metrics]
ENABLED = true
TOKEN = \${GITEA_METRICS_TOKEN}

[security]
INSTALL_LOCK = true
PASSWORD_COMPLEXITY = lower,upper,digit,spec
MIN_PASSWORD_LENGTH = 14
PASSWORD_CHECK_PWN = true             ; HIBP check

[service]
ENABLE_CAPTCHA = true
CAPTCHA_TYPE = hcaptcha
REQUIRE_SIGNIN_VIEW = true

[oauth2_client]
ENABLE_AUTO_REGISTRATION = false

[session]
PROVIDER = redis
PROVIDER_CONFIG = addr=redis:6379,db=1

[cache]
ADAPTER = redis
HOST = addr=redis:6379,db=2

[queue]
TYPE = redis
CONN_STR = redis://redis:6379/3

[log]
MODE = console, file
LEVEL = Info
ROUTER = console
```

Branch Protection (per default template):

- ✅ Disable force push
- ✅ Disable branch deletion
- ✅ Require signed commits (optional, recommended)
- ✅ Require pull-request review
- ✅ Require status checks (CI green)

- ✅ Block pushes that introduce secrets (enforced by pre-receive hook integration — see §9.4)

Alt forge: Forgejo — drop-in replacement, same API, community-governed, recommended for teams that prefer non-corporate governance. `[FACT-CHECKED]`

4.2 Event Bus — Redis Streams

Role: Durable, ordered, at-least-once delivery of push events from Webhook Receiver to the Smart-Worker. Decouples Gitea from fan-out logic.

Why Redis Streams (not Lists, not Pub/Sub):

- **Durable:** With `appendonly yes`, events survive crashes.
- **Consumer groups:** Enable horizontal scaling across multiple Smart-Worker pods (XREADGROUP + PEL).
- **Replay:** Events remain for inspection/replay via XREAD.
- **Back-pressure:** `XLEN` queue depth becomes a first-class metric.
- **Lightweight:** ~5 MB RAM for 10^5 events.

Stream design:

```
Stream per (repo_id, upstream_name) pair:  gitpx:events:{repo_id}:{upstream}
Consumer group per upstream:                gitpx:workers:{upstream}
Control channel (pub/sub):                  gitpx:control
```

Each push event = one entry:

```
{
  "event_id":    "01HQ7M6X...",          // ULID
  "repo_id":     "123",
  "repo_name":   "acme/my-project",
  "ref":         "refs/heads/main",
  "before_sha":  "0000...",
  "after_sha":   "a1b2c3...",
  "pusher":      "milos",
  "pusher_email":"milos@example.com",
  "commits":     [ /* commit metadata */ ],
  "ts":          "2026-04-19T15:02:11Z",
  "webhook_hmac":"sha256=..."            // verified at ingress
}
```

4.3 Execution Layer — Go Smart-Worker

Role: Consume events, clone/update a temporary bare repo, push to every configured upstream, handle retries, rate limits, LFS, provenance, and metrics.

Tech stack:

- **Language:** Go 1.22+ `[FACT-CHECKED as current stable]`
- **Libraries:**
  - `github.com/go-git/go-git/v5` — Git operations
  - `github.com/redis/go-redis/v9` — Redis client
  - `github.com/prometheus/client_golang` — metrics
  - `github.com/sigstore/cosign/v2` — commit signing
  - `github.com/sethvargo/go-retry` — backoff with jitter
  - `github.com/spf13/viper` — config (also supports hot-reload)
  - `golang.org/x/sync/errgroup` — parallel pushes with bounded concurrency
  - `go.uber.org/zap` — structured logging

High-level processing loop:

```go
// pseudo-code — see §17 Appendix B for full reference
for {
    msgs, _ := rdb.XReadGroup(ctx, &redis.XReadGroupArgs{
        Group:    "gitpx:workers:github",
```

```
        Consumer: hostname,
        Streams:  []string{streamKey, ">"},
        Count:    1,
        Block:    5 * time.Second,
    }).Result()

    for _, stream := range msgs {
        for _, m := range stream.Messages {
            evt := parseEvent(m.Values)
            cfg := loadUpstream(evt.RepoName, "github")   // hot-reloaded
            if err := pushToUpstream(evt, cfg); err != nil {
                observeFailure(err, cfg)
                retryWithBackoff(evt, cfg, err)
                continue
            }
            attestWithSigstore(evt, cfg)
            rdb.XAck(ctx, streamKey, group, m.ID)
            observeSuccess(cfg)
        }
    }
}
```

4.4 The Universal Git Adapter `[RESOLVED]`

This resolves the **most consequential inconsistency** between `Git_Proxy_Idea_Fixed.md` (push without `--force`) and `Git_Proxy_Idea_Fixed_2.md` (push **with** `--force`).

Definitive policy:

1. **Never use `git push --mirror`.** It deletes refs on the destination that don't exist on the source — catastrophic in the presence of concurrent force-pushes or temporary state divergence.
2. **Use explicit refspecs:** `refs/heads/*:refs/heads/*` and `refs/tags/*:refs/tags/*`. This never deletes refs on the remote; it only creates/updates.
3. **For non-fast-forward updates (intentional force-push on a branch explicitly marked `allow_force: true` in config),** use `--force-with-lease=<ref>:<expected_sha>`. This fails cleanly if the remote's current tip differs from what the Smart-Worker expected — preventing races that clobber a concurrent push made by a human directly to the upstream.
4. **Never use plain `--force`** — it is unsafe in the multi-proxy-observer scenario.
5. **Deletion of refs on upstreams must be an explicit, logged, audited operation** — never implicit. A separate `delete_ref` event in the stream triggers the Smart-Worker to issue `git push <remote> :refs/heads/<branch>`, and this is denied by default for any branch marked as protected in Gitea.

Canonical push command generated by the Smart-Worker:

```
# Fast-forward / new refs — default path
git push "\$AUTH_URL" \
    'refs/heads/*:refs/heads/*' \
    'refs/tags/*:refs/tags/*'

# Intentional force-update on a branch with allow_force=true
git push --force-with-lease="refs/heads/\${BRANCH}:\${EXPECTED_SHA}" \
    "\$AUTH_URL" \
    "refs/heads/\${BRANCH}:refs/heads/\${BRANCH}"
```

This policy is **non-negotiable** and supersedes both source documents.

4.5 Dynamic Upstream Configuration

Configuration lives in **two tiers**:

- **Tier 1 (global defaults):** `/etc/gitpx/upstreams.yaml` — version-controlled in its own Git repo (managed by the proxy itself — recursive dog-fooding).
- **Tier 2 (per-repo override):** stored in the Gitea repo under `.gitpx/upstreams.yaml` on the default branch (read by the Webhook Receiver on each push).

Schema:

```yaml
apiVersion: gitpx.io/v1
kind: UpstreamSet
metadata:
  repo: acme/my-project          # glob supported (e.g., "acme/*")
spec:
  upstreams:
    - name: github
      url: https://github.com/acme/my-project.git
      enabled: true
      branches: ["*"]            # glob
      tags: true
      lfs: true
      token_env: PASSWORD_GITHUB
      allow_force: false
      rate_limit_strategy: github_v4     # see §6.3
      priority: 10               # lower = processed first
      timeout: 60s
      retry:
        max_attempts: 8
        initial_delay: 1s
        max_delay: 10m
        jitter_pct: 25
      attestation:
        sigstore: true
        slsa_level: 3

    - name: gitlab
      url: https://gitlab.com/acme/my-project.git
      enabled: true
      token_env: PASSWORD_GITLAB
      rate_limit_strategy: gitlab_headers
      priority: 20

    - name: gitee
      url: https://gitee.com/acme/my-project.git
      token_env: PASSWORD_GITEE
      rate_limit_strategy: constant_backoff
      geography: cn              # hint for route optimization
      priority: 30

    - name: gitverse
      url: https://gitverse.ru/acme/my-project.git
      token_env: PASSWORD_GITVERSE
      rate_limit_strategy: generic
      geography: ru
      priority: 40

    - name: gitflic
      url: https://gitflic.ru/acme/my-project.git
```

```yaml
      token_env: PASSWORD_GITFLIC
      rate_limit_strategy: generic
      geography: ru
      priority: 50

    - name: bitbucket
      url: https://bitbucket.org/acme/my-project.git
      token_env: PASSWORD_BITBUCKET
      rate_limit_strategy: bitbucket_headers
      priority: 60

    - name: codeberg
      url: https://codeberg.org/acme/my-project.git
      token_env: PASSWORD_CODEBERG
      rate_limit_strategy: generic
      priority: 70

    - name: internal-mirror
      url: https://internal.corp/acme/my-project.git
      token_env: PASSWORD_INTERNAL
      rate_limit_strategy: none
      priority: 5                # push before public forges
```

Hot-reload mechanism:

The Smart-Worker watches `upstreams.yaml` via `fsnotify` **and** subscribes to the `gitpx:control` Redis Pub/Sub channel. When the file changes, or when a `RELOAD` message is published (e.g., by the Web UI's "Apply" button), the new config is validated (JSON-schema) and atomically swapped into active memory. **No restart. No dropped events.**

---

5. Smart-Worker — Deep Implementation

5.1 Binary Layout

Single static binary, cross-compiled for `linux/amd64`, `linux/arm64`, `linux/riscv64`, `darwin/arm64`, `windows/amd64`. Distributed via:

- Docker multi-arch manifest on `ghcr.io/gitpx/smart-worker`
- Checksummed, cosign-signed release artifacts on GitHub Releases
- Homebrew tap, `apt` (via `deb-s3`), `rpm` (via `rpm-s3`), Nix flake, Arch AUR

5.2 Internal Package Layout

```
cmd/
  worker/           # main entrypoint
  receiver/         # webhook receiver entrypoint
  cli/              # gitpxctl admin CLI
internal/
  config/           # config loading, hot-reload, validation
  events/           # GitEvent parsing, serialization
  redis/            # Stream consumer, XPENDING reaper
  gitops/           # go-git wrappers, safe push logic
  adapters/         # per-provider strategies
    github/
    gitlab/
    gitee/
    gitflic/
    gitverse/
    bitbucket/
    codeberg/
    generic/
  ratelimit/        # token bucket, header parsing
  lfs/              # LFS handling, Dragonfly client
  attest/           # Sigstore/Cosign/Rekor
  secrets/          # env-var credential provider, ptrace-safe
  metrics/          # Prometheus instrumentation
  health/           # liveness, readiness, diagnostics
  plugin/           # WASM plugin host
```

```
pkg/
  sdk/              # public Go SDK for external plugins
```

5.3 Concurrency Model

- **One goroutine per active upstream per repo** (bounded by `semaphore.NewWeighted(N)` where N = total CPU × 2).
- **Per-remote token bucket rate-limiter** (`golang.org/x/time/rate`).
- **Per-remote circuit breaker** (`github.com/sony/gobreaker`): after 5 consecutive failures, the remote is quarantined for 60 s, metric `gitpx_circuit_open{remote}` is set, and an alert fires.
- **Graceful shutdown:** SIGTERM causes the worker to stop pulling new events, drain the in-flight WaitGroup, `XACK` only completed messages, and exit with code 0.
- **Idempotency:** The worker may re-receive a message (at-least-once semantics). Before doing any work, it consults a Redis `SETNX gitpx:lock:{event_id}` with 10-minute TTL. If already held, the message is logged and acked (skipped).

5.4 Ephemeral Working Directory

The worker does **not** hold a persistent clone of every repo. Instead, for each event:

1. Shallow-fetch the required ref into an ephemeral bare repo at `/var/lib/gitpx/tmp/{event_id}.git` using `git clone --bare --filter=tree:0 --no-checkout <gitea_url>`.
2. Configure the upstream remote on that ephemeral clone.
3. Execute `git lfs fetch --all` if the repo uses LFS.
4. Execute the safe-push command from §4.4.
5. Delete the ephemeral clone on success or on terminal failure.

Why ephemeral? Zero state drift, zero disk pressure for big monorepos, no risk of a previous corrupted checkout affecting a new push, parallelism is trivial (just use different temp dirs).

Optimisation: For hot repos (> 5 events/minute), the worker can maintain a warm bare clone at `/var/lib/gitpx/warm/{repo_id}.git` and `git fetch` into it before pushing. Controlled by a per-repo `warm_cache: true` config flag.

---

6. Universal Git Adapter + Provider-Specific Recipes

6.1 The Adapter Interface

```go
type UpstreamAdapter interface {
    Name() string
    Push(ctx context.Context, evt *GitEvent, cfg *UpstreamConfig) error
    DeleteRef(ctx context.Context, ref string, cfg *UpstreamConfig) error
    RateLimiter() ratelimit.Limiter
    Healthcheck(ctx context.Context, cfg *UpstreamConfig) error
    SupportsLFS() bool
    Capabilities() Capabilities
}
```

The `generic` **adapter** implements the universal HTTPS push that works against ANY Git server (the "basic generic working system" required by the original spec). Specialized adapters override behaviors where providers expose APIs (e.g., rate-limit headers, repo creation, merge-request status updates).

6.2 Provider Matrix

Provider	Token Type	Rate-Limit Strategy	Repo Auto-Create	Special Notes
GitHub	PAT (classic or fine-grained), GitHub App	Parse `X-RateLimit-*` headers	✅ via REST `POST /user/repos`	LFS works out-of-box; supports `git-lfs-transfer` v2
GitLab	PAT (scope `write_repository`) or OAuth	Parse `RateLimit-*` headers	✅ via REST `POST /projects`	LFS native; supports deploy tokens
Gitee	Private Token	Constant backoff (no reliable headers)	✅ via REST `POST /user/repos`	Regional egress may be slow from US/EU — route via Cloudflare Argo if needed

Provider	Token Type	Rate-Limit Strategy	Repo Auto-Create	Special Notes
GitFlic	PAT	Generic backoff	⚠️ Manual (no stable create API as of writing)	Russian-hosted; check geo-reachability
GitVerse	PAT (SberGit)	Generic backoff	⚠️ Manual	Sber-owned; API stability improving
Bitbucket Cloud	App Password or Workspace Access Token	Parse `X-RateLimit-*` headers	✅ via REST 2.0	LFS requires premium plan
Codeberg (Gitea)	PAT	Generic (Gitea rate limit is lenient)	✅ via Gitea API	Community-funded; respect their infrastructure
Forgejo (any)	PAT	Generic	✅ via Forgejo API	Protocol identical to Gitea
SourceHut	Personal OAuth2 token	Generic	✅ via GraphQL API	Requires SSH push preferred
Azure DevOps	PAT	Generic	✅ via REST	SSH or HTTPS with `gcm` auth
AWS CodeCommit	IAM signed HTTPS (v4)	Generic	✅ via CLI/SDK	Requires `git-remote-codecommit` helper
Generic (any)	Basic auth / token-in-URL	Constant backoff	❌	Fallback; always works if server speaks HTTPS smart-HTTP

6.3 Rate-Limit Strategies

- `github_v4`: On every push, after the HTTP response is available, parse `X-RateLimit-Remaining`, `X-RateLimit-Reset`. If `Remaining < 50`, sleep until `Reset` + 5 s jitter.
- `gitlab_headers`: Same pattern, headers are `RateLimit-Remaining`, `RateLimit-Reset`.
- `bitbucket_headers`: `X-RateLimit-Remaining`, `X-RateLimit-Reset` (epoch seconds).
- `constant_backoff`: 500 ms between pushes for providers without headers.
- `generic`: No preemptive throttling; rely on 429 → backoff.
- `none`: No limiting (internal mirrors).
- `adaptive`: `[NEW]` Maintains an EWMA of response latencies; auto-throttles when p95 latency exceeds 2× baseline (suggests upstream is under load).

6.4 Plugin Architecture for New Providers `[NEW]`

Adding a new Git service requires **no recompilation** thanks to a WASM plugin system:

```
type PluginHost struct {
    rt   wazero.Runtime
    mods map[string]api.Module
}

// Plugin ABI (functions exported by WASM module):
//   push(event_ptr, event_len, config_ptr, config_len) -> error_ptr, error_len
//   healthcheck(config_ptr, config_len) -> error_ptr, error_len
//   rate_limit(headers_ptr, headers_len) -> wait_ms
```

Plugins can be written in **Rust, Go (TinyGo), AssemblyScript, C, Zig** — compiled to `*.wasm`, dropped into `/etc/gitpx/plugins/`, and hot-loaded. `[NEW]`

6.5 Alternative Fallback: Bash `post-receive` Hook

For **air-gapped single-host deployments** where even Redis is unavailable, a reference bash implementation is provided as a fallback. This is the solution described in the original `Git_Proxy_Idea.md`. It is kept as **`contrib/fallback/post-receive.sh`** — documented, tested, but **not recommended for production**. Its known limitations:

- No retry queue (push fails = push lost unless manually replayed).
- Serial fan-out (slow for 5+ upstreams).
- Secrets in environment variables only (no vault).

Use only when the full architecture is impossible.

---

7. Game-Changing Innovations (Extended)

This section consolidates the innovations from the source documents and adds the `[NEW]` ones requested.

7.1 Zero-Trust Network Access via Cloudflare Tunnels

The Oracle VM has **zero** public ingress ports. `cloudflared` runs as a Docker container and establishes an **outbound-only** persistent HTTP/2 tunnel to Cloudflare's edge. All inbound traffic arrives from Cloudflare, where WAF, DDoS protection, bot management, and TLS termination occur for free.

Benefits:

- Immune to SSH brute-force and port scanners.
- Free TLS (no Let's Encrypt plumbing, no cert rotation).
- Free DDoS absorption at edge.
- Geographic routing via Cloudflare's anycast network.
- Optional Cloudflare Access for zero-trust authentication before even reaching Gitea.

7.2 Transparent LFS Proxying via Dragonfly

Problem: Git LFS objects are not transferred via `git push` — they require the LFS client and frequently fail under network flakiness or cert issues (R-008).

Solution: Deploy **Dragonfly** (CNCF project; P2P file distribution) alongside Gitea. The Smart-Worker calls `git lfs push --all <remote>` explicitly, with LFS URL rewritten to flow through Dragonfly's HTTP proxy. Dragonfly chunks large files, uses P2P between worker instances, and retries transparently. `[FACT-CHECKED]`

7.3 Cryptographic Provenance via Sigstore + Cosign + Rekor

On successful push to each upstream, the Smart-Worker:

1. Generates a keyless signature (via `cosign sign-blob --yes`) over `{repo, commit_sha, upstream, timestamp}` JSON.
2. Uploads the signature to the **Rekor** public transparency log.
3. Stores the Rekor entry index as a Git note (`refs/notes/gitpx-attest`) on the commit.

Result: **mathematical proof** that a given commit SHA was deployed from the proxy to a given upstream at a given time. Auditors can verify independently via the public Rekor instance — no trust in the proxy operator required.

7.4 Automated Secret Annihilation

The **Webhook Receiver** is a **blocking** gate:

1. On webhook arrival, HMAC is verified (rejects forgeries).
2. The diff is scanned with **Gitleaks** (and optionally **TruffleHog** / **detect-secrets** in parallel for defense-in-depth).
3. If a secret is detected:
    - The Redis event is **not** written (fan-out is aborted).
    - Gitea API is called to lock the repository (`PUT /repos/{owner}/{repo}`).
    - Admin is alerted via multi-channel (Telegram, Slack, email, PagerDuty).
    - An incident ticket is auto-created in Gitea Issues.
4. If clean, the event is written to Redis Streams.

This gate ensures a secret **never** propagates to public mirrors — even if the developer bypasses pre-commit hooks.

7.5 SBOM & License Compliance `[NEW]`

On every push, the Webhook Receiver also runs:

- **Syft** — generates CycloneDX/SPDX Software Bill of Materials.
- **Grype** — CVE scan against the SBOM.
- **ORT (OSS Review Toolkit)** — license compliance scan.

Results are stored as Git notes (`refs/notes/gitpx-sbom`, `refs/notes/gitpx-cve`, `refs/notes/gitpx-license`) and surfaced in the Gitea repo's "Insights" tab via a custom middleware.

7.6 Deterministic Reproducible Builds Proof `[NEW]`

Optionally, the Smart-Worker can trigger a **hermetic build** (via Nix or Bazel RBE) from the commit and attach a SLSA L3 provenance attestation. This turns the proxy into a **build-attestation service** as well as a mirror.

7.7 Federated Event Streaming `[NEW]`

Multiple `gitpx` instances (across geographies or organizations) can federate via an NATS JetStream bridge. A push accepted by the Belgrade instance can be replicated to the Tokyo instance's canonical store, making the proxy itself highly available across regions.

7.8 GitOps for the Proxy Itself `[NEW]`

The proxy's own configuration (`upstreams.yaml`, `app.ini`, Caddyfile, Prometheus rules) lives in a Git repo — managed by the proxy itself. Changes are proposed via PR, reviewed, merged; a GitOps reconciler (Flux/ArgoCD Lite) applies them. Recursive, delightful, and auditable.

7.9 Cryptographic Time-Locked Backups `[NEW]`

Backups uploaded to Backblaze B2 are encrypted with a **time-lock puzzle** (Rivest's LCS35) — decryption requires a bounded amount of sequential computation, preventing rapid-exfiltration even if backup credentials leak.

7.10 Commit Attestation via Sigstore Fulcio + Gitsign `[NEW]`

Incoming pushes can be verified: if `gitsign`-signed, the signature is verified against Fulcio before acceptance. This adds an optional but strong layer: only signed commits accepted.

7.11 AI-Powered "Intent Diff" Summaries `[NEW]`

On each push, an LLM generates a 1-paragraph "what changed and why" summary, stored as `refs/notes/gitpx-summary/<sha>`. Developers see it in the dashboard. Useful for audits, onboarding, code archaeology.

7.12 Real-Time P2P Developer Awareness `[NEW]`

A WebRTC mesh between online developer clients (via the Web dashboard or desktop app) broadcasts "who is viewing which repo/branch" and "who is currently pushing what", similar to Figma's presence. Powered by a free TURN-less STUN config + a small NATS signaling channel.

7.13 OpenCV-Powered Visual Regression for Docs Sites `[NEW]`

If the repo hosts a documentation site (detected by presence of `docs/` or `mkdocs.yml`), the proxy runs a headless Playwright crawl on key pages, stores screenshots, and uses OpenCV SSIM diffs to flag visual regressions in PRs.

7.14 CUDA-Accelerated Fingerprint Deduplication `[NEW]`

Large-repo push detection uses CUDA-accelerated rolling-hash fingerprinting (via `libhashcat` kernels) to identify identical content across forks, reducing LFS bandwidth by up to 90%.

7.15 eBPF-Based Micro-Observability `[NEW]`

An eBPF program (loaded via `cilium/ebpf` Go library) attached to Gitea's and Smart-Worker's processes produces kernel-level traces of syscalls, TCP connects, file I/O. Surfaced through Pixie-style flame graphs in Grafana.

---

8. Comprehensive Risk Landscape (CIA Model, Extended)

All identified risks — extended from the 10 in source documents to **25** — each with category, likelihood, impact, owner, and mitigation (§9).

ID	Cat	Risk	Likelihood	Impact	Owner	Mitigation §
R-001	Integrity	Corrupted source of truth (disk/RAM/admin error)	Low	Catastrophic	SRE	9.1
R-002	Integrity	Divergent/corrupted mirrors via `--mirror` or plain `--force`	Medium	High	Backend	9.2, 4.4
R-003	Integrity	Partial push state across upstreams	High	High	Backend	9.3
R-004	Confidentiality	Secret leakage via Git history	High	Critical	Security	9.4

ID	Cat	Risk	Likelihood	Impact	Owner	Mitigation §
R-005	Availability	Mirror queue starvation	Medium	Medium	Backend	9.5
R-006	Availability	Upstream rate limiting (HTTP 429)	High	Medium	Backend	9.6
R-007	Availability	Gitea outage / host crash	Low	Critical	SRE	9.7
R-008	Integrity	LFS sync failure	Medium	Medium	Backend	9.8
R-009	Availability	Token expiry/revocation silent drift	High	High	SRE	9.9
R-010	Integrity	Redis/Worker data loss mid-processing	Low	High	Backend	9.10
R-011	Confidentiality	Webhook forgery / replay attacks `[NEW]`	Medium	High	Security	9.11
R-012	Integrity	Dependency supply-chain compromise `[NEW]`	Medium	High	Security	9.12
R-013	Availability	Cloudflare Tunnel outage `[NEW]`	Low	High	SRE	9.13
R-014	Confidentiality	Backblaze B2 credentials leak → ransomware `[NEW]`	Low	Catastrophic	Security	9.14
R-015	Integrity	Timezone/clock skew causing signature rejection `[NEW]`	Low	Medium	SRE	9.15
R-016	Availability	DNS hijacking / MITM at upstream `[NEW]`	Low	Critical	Security	9.16
R-017	Privacy	LLM prompt leakage of proprietary code `[NEW]`	High	High	AI/Security	9.17
R-018	Cost	LLM free-tier exhaustion mid-incident `[NEW]`	High	Low-Med	AI	9.18
R-019	Availability	Host disk full (Git LFS, logs, backups) `[NEW]`	Medium	High	SRE	9.19
R-020	Integrity	Git submodule / subtree desync `[NEW]`	Medium	Medium	Backend	9.20
R-021	Integrity	Large-object corruption during transfer `[NEW]`	Low	High	Backend	9.21
R-022	Legal/Compliance	Pushing export-controlled code to jurisdictions that reject it `[NEW]`	Medium	Catastrophic	Legal	9.22
R-023	Availability	SSH host-key mismatch on upstream causing push refusal `[NEW]`	Medium	Medium	Backend	9.23
R-024	Integrity	Replay of old stream events after Redis restart `[NEW]`	Low	Medium	Backend	9.24
R-025	Availability	Client-side app bugs corrupting local working copy (Android/iOS/etc.) `[NEW]`	Medium	Low	Mobile	9.25

---

9. Defense-in-Depth Mitigations (Per Risk)

9.1 R-001 — Source of Truth Integrity

- **Hourly:** `restic` incremental backup of `/var/lib/gitpx/gitea` and Postgres logical dump to Backblaze B2 with Object Lock (compliance mode, 30-day retention minimum).
- **Daily:** `gitea dump --config /data/gitea/conf/app.ini` to a second backup target (e.g., Oracle Object Storage).

- **Weekly:** Full `pg_dumpall` tested by automated restore into a throwaway container in CI.
- **ZFS snapshots** on the host (if ZFS is available): every 15 min, retained for 24 h.
- **Monthly DR drill:** Provision a fresh VM in another region, restore from B2, run E2E smoke tests. Drill report auto-filed as a Gitea Issue.
- **BTRFS/ZFS scrub** to detect silent disk corruption.
- **ECC memory** on the VM (Oracle A1.Flex provides ECC by default).

9.2 R-002 — Mirror Divergence

- **Forbid `git push --mirror`** in Smart-Worker code. Linted by `go vet` custom analyzer.
- **Force-push** only via `--force-with-lease=<ref>:<expected_sha>` (see §4.4), and only for branches with explicit `allow_force: true` in the upstream config.
- **Protected branch list** mirrored from Gitea; Smart-Worker refuses to delete or force-push any branch marked protected.

9.3 R-003 — Partial Push State

- **Per-remote Redis Streams** — each upstream has its own independent queue and consumer group.
- **Drift detector:** daily cron job calls `git ls-remote <each_upstream>` and compares tips to Gitea's HEADs. Any drift → inject a resync event.
- **Metrics:** `gitpx_upstream_drift_total{remote}` gauge, alert `UpstreamDrift` fires if > 0 for > 30 min.
- **Idempotent retries** — safe to replay the same event N times.

9.4 R-004 — Secret Leakage

- **Pre-commit client hooks:** bundled repo template includes `.pre-commit-config.yaml` with `gitleaks` and `detect-secrets`.
- **Server-side pre-receive hook:** a Go binary in `/data/gitea/git/hooks/pre-receive.d/` runs `gitleaks protect --staged` against the push payload and rejects (exit 1) if a secret is detected. This is enforced at Gitea level, before the commit is even accepted to the bare repo.
- **Post-receive Webhook Receiver** re-scans as belt-and-suspenders.
- **Token-scanning alerts** registered with GitHub Secret Scanning API (free for public repos).
- **Automated rotation:** if a leak is confirmed, integration with Hashicorp Vault / AWS Secrets Manager / Bitwarden triggers rotation of the compromised credential.

9.5 R-005 — Queue Starvation

- **Per-remote consumer groups** — a stuck upstream cannot block others.
- **XPENDING reaper:** goroutine scans for messages idle > 5 min in the PEL, reclaims them for another consumer, after 3 reclaims → DLQ (`gitpx:events:dlq`).
- **Dead-letter queue** has its own dashboard + alert.

9.6 R-006 — Rate Limiting

- **Per-provider rate limiter** with header awareness (§6.3).
- **Exponential backoff with jitter:** `delay = min(max_delay, initial_delay * 3^attempt) ± jitter_pct%`.
- **Adaptive throttling** based on EWMA latency.
- **Batching:** if 3 pushes to the same ref arrive within 500 ms, they are coalesced (final SHA wins).

9.7 R-007 — Gitea Outage

- **Docker `restart: always`** for all services.
- **Systemd watchdog** at the host level.
- **Healthcheck endpoint** probed by Cloudflare; on 3 consecutive failures, a Telegram alert + automated `docker compose restart gitea` is triggered by a watchdog goroutine in the Smart-Worker.
- **For serious outages:** The DR playbook provides restore-to-new-VM in ≤ 60 min.
- **Read replica [FUTURE]:** At scale, a Postgres streaming replica in a second region enables fast failover.

9.8 R-008 — LFS Failure

- **Dragonfly proxy** (see §7.2) for reliable chunked transfer.
- **Explicit `git lfs fetch --all` then `git lfs push --all`** in the worker's push routine, before the Git packfile push.
- **LFS object hash verification** after push via `git lfs fsck`.
- **Healthcheck:** periodic `git lfs ls-files` comparison between Gitea and each LFS-enabled upstream.

9.9 R-009 — Token Expiry

- **Token lifetime tracking:** the admin UI shows expiry dates for all tokens (scraped from provider APIs where available).
- **Pre-expiry alerts** at 30/14/7/1 day thresholds.
- **Automated rotation via provider APIs** where supported (GitHub fine-grained tokens, GitLab, Bitbucket).

- **Silent-drift canary:** the drift detector (§9.3) catches silent auth failures even when the worker's logs aren't being watched.

9.10 R-010 — Mid-Processing Data Loss

- **Redis AOF (`appendonly yes`, `appendfsync everysec`).**
- **At-least-once semantics** via `XREADGROUP` + explicit `XACK` only on success.
- **PEL reaping** for crashed consumers (§9.5).
- **Event persistence** — events are kept in the stream for 7 days (`XTRIM MAXLEN ~` with `MINID` threshold).

9.11 R-011 — Webhook Forgery `[NEW]`

- **HMAC-SHA256** secret shared between Gitea and the Receiver, verified on every request.
- **Timestamp header** `X-Gitpx-Timestamp` checked against server clock; reject > 5 min skew.
- **Nonce cache** in Redis prevents replays (`SETNX gitpx:nonce:{hash} 900`).
- **Source IP allowlist** (Cloudflare internal ranges only).

9.12 R-012 — Supply-Chain `[NEW]`

- **Go module proxy pinning** (`GOPROXY=direct`).
- **Vendored dependencies** in the release build.
- **SLSA L3 build provenance** attached to every release.
- `govulncheck` on every CI run.
- **Dependency renewal** via Renovate bot; no automatic merges without human review for security-sensitive packages.

9.13 R-013 — Cloudflare Tunnel Outage `[NEW]`

- **Dual-tunnel failover** — a second `cloudflared` replica with a different Cloudflare account acts as hot standby.
- **Caddy fallback mode** — on tunnel outage, Caddy can temporarily expose port 443 with Let's Encrypt; toggled by `gitpxctl fallback enable`.
- **Tailscale standby** — a Tailscale subnet router provides out-of-band admin access if the tunnel is down.

9.14 R-014 — Backup Credentials Leak `[NEW]`

- **Restic repository password** stored in a local file with 0600 perms, owned by a dedicated backup user.
- **Backblaze B2 Application Key** scoped to write-only + bucket-specific + no-list-files.
- **Object Lock in compliance mode** — even B2 admin cannot delete backups within retention.
- **Periodic backup integrity verification** via `restic check --read-data-subset`.

9.15 R-015 — Clock Skew `[NEW]`

- `chrony` bound to multiple NTP sources (pool.ntp.org, time.cloudflare.com, time.google.com).
- **Monotonic clock** for token-bucket accounting.
- **Sigstore signature timestamp** tolerance of ±5 min verified on issuance.

9.16 R-016 — DNS Hijack / MITM `[NEW]`

- **DoH (DNS-over-HTTPS)** configured on the host via `dnscrypt-proxy` → Cloudflare / Quad9.
- **SSH known-hosts** pinned per upstream; push aborts on mismatch.
- **Certificate pinning** for critical upstreams via custom HTTP transport in Go.
- **DNSSEC** validation where available.

9.17 R-017 — LLM Prompt Leak `[NEW]`

- **Diff redaction filter** — before sending to any cloud LLM, regex-redact lines matching secret patterns (even if Gitleaks missed them).
- **Private-mode repos** — users can mark a repo `ai_external: false`; only local Ollama is used for those repos.
- **Zero-retention contracts** where available (OpenAI ZDR, Anthropic ZDR).
- **Prompt audit log** stored locally; user can export what was ever sent to which provider.

9.18 R-018 — LLM Quota Exhaustion `[NEW]`

- **Cascade of providers** (Gemini → Groq → OpenRouter free → Ollama local).
- **Priority tiers:** critical alerts always fall back to Ollama; nice-to-have features fail silently on quota out.

9.19 R-019 — Disk Full `[NEW]`

- **Per-mount disk-pressure alerts** at 70/85/95 % thresholds.

- **Log rotation** via `logrotate` + `journald` vacuum.
- **LFS GC** schedule: `git lfs prune --verify-remote` weekly.
- **Backup rotation** retention enforced (keep-daily=7, keep-weekly=5, keep-monthly=12).
- **Emergency purge script** (`gitpxctl disk emergency-purge`) that aggressively vacuums logs, prunes LFS, rotates backups.

9.20 R-020 — Submodule Desync `[NEW]`

- **Submodule-aware push:** when a commit touches a `.gitmodules`, the worker logs a warning and (optionally) pushes the submodule repos first if they are also managed by the proxy.
- **Recursive fetch** during the ephemeral clone (`--recurse-submodules`).

9.21 R-021 — Object Corruption in Transit `[NEW]`

- `git fsck --full` on the ephemeral clone after fetch and before push.
- **Pack verification** — `git verify-pack -v` on the fetched packfile.
- **Post-push validation:** `git ls-remote --exit-code` confirms the ref reached the upstream.

9.22 R-022 — Export Control `[NEW]`

- **Per-repo geographic allowlist/denylist** in config (`geo_allow: ["eu", "us"]` or `geo_deny: ["xx"]`).
- **Upstream geography metadata** (§4.5 `geography:` field).
- **Legal review tag** — repos tagged `export_controlled: true` cannot be auto-enabled against new upstreams; requires explicit admin approval in the dashboard.

9.23 R-023 — SSH Host-Key Drift `[NEW]`

- **Pinned known_hosts** bundled with the worker image.
- **On mismatch:** push fails cleanly, operator alerted, `gitpxctl trust-hostkey <upstream>` required to re-pin.

9.24 R-024 — Stream Replay After Restart `[NEW]`

- **Idempotency lock** (§5.3) prevents reprocessing.
- **Event TTL** — events older than 24 h are skipped with a warning.

9.25 R-025 — Client Bug Corrupting Local WC `[NEW]`

- **Mobile/desktop clients never write to user's `.git/` directory** without a dry-run + confirmation step.
- **Immutable clone-and-work** mode: client-side operations go through a sandbox overlay FS.
- **Crash reporting** via Sentry (free tier) with auto-redaction of file paths.

---

10. AI / LLM Integration Strategy

10.1 Philosophy

LLMs are used as **augmentation**, not automation of critical paths. Every agent is:

- **Read-only by default** (cannot merge PRs, cannot push, cannot delete).
- **Scoped** — a PR reviewer sees only the diff, not the entire repo.
- **Auditable** — every prompt and response is logged to Loki.
- **Replaceable** — any agent can be disabled without affecting the proxy's core function.
- **Failsafe** — agents never block pushes; their output is advisory.

10.2 Agent Roles

Agent	Trigger	Input	Output	Authority	Model Class
PR Reviewer	`pull_request.opened`, `.synchronize` webhook	Diff (≤ 8k tokens)	Inline comments, severity	Comment only	Code-specialized (Qwen2.5-Coder, DeepSeek-Coder, Groq Llama)
Patch Proposer	Issue with label `ai-assist`	Issue body + repo context	New branch + draft PR	Opens PR, no merge	Code model with tool use
Security Sentinel	Scheduled (daily) + `push` to `main`	Full codebase (RAG)	New issues, CVE refs	File issue only	Code + security-fine-tuned model

Agent	Trigger	Input	Output	Authority	Model Class
Observability Interpreter	Alertmanager fires	Alert + recent logs/metrics	Plain-English root-cause summary, suggested action	Read-only ops data	General (Gemini Flash, Llama 3.3)
Incident Responder	Gitleaks trigger, service crash	Context + runbook	Allowlisted actions (pause queue, revoke session, revoke token)	Pre-approved actions only	General + tool use
Commit Summarizer	Every push	Commit message + diff	1-sentence plain-English summary, stored as Git note	Write to `refs/notes/gitpx-summary`	Small general model
Docs Writer	`.md` / `.rst` changes	Diff + docs context	Suggested improvements in PR comment	Comment only	General model
Dependency Upgrader	Weekly cron	Lockfiles + CVE feed	PR upgrading deps	Opens PR, no merge	Code model
Refactor Suggester `[NEW]`	`ai-refactor` label on issue	Target file + surrounding context	Proposed refactor in branch	Opens PR	Code model
Test Generator `[NEW]`	PR with missing tests (detected by coverage drop)	Production code of the PR	Suggested tests in PR comment	Comment only	Code model
Onboarding Guide `[NEW]`	First push from a new user	User profile + repo README	Personalized welcome message / DM	Sends via bot integration	General model
Release Notes Drafter `[NEW]`	New tag pushed	Commit range between tags	Markdown release notes draft	Creates Release draft	General model
Triage Bot `[NEW]`	New issue	Issue body + repo taxonomy	Suggested labels, assignees	Apply labels	General model
Architecture Sentinel `[NEW]`	Changes touching `/cmd`, `/internal`	Diff + architecture doc	Warns on architecture violations	Comment only	Code model

10.3 Zero-Cost LLM Provider Matrix `[VERIFY-AT-INTEGRATION]`

Free-tier details change frequently. The cascading strategy insulates the system from single-provider policy changes.

Provider	Notable Models	Access	Typical Free Rate	Integration
Google AI Studio (Gemini)	Gemini 2.5 Pro/Flash/Flash-Lite	API key, no credit card	Daily cap on free tier (varies)	OpenAI-compatible endpoint
Groq	Llama 3.3 70B, Kimi K2, others	API key	High RPM, capped RPD	OpenAI-compatible, very fast (≈500 tok/s reported)
OpenRouter	40+ free models (Qwen, DeepSeek R1, Gemma, etc.)	API key	20 RPM / 200 RPD on free tier	OpenAI-compatible gateway
Together AI	GPT-OSS, Llama, Qwen, DeepSeek	Free credits on signup	Free credits then paid	OpenAI-compatible
DeepSeek	DeepSeek-Chat, DeepSeek-R1	API key	Paid (very cheap)	OpenAI-compatible
Cerebras	Llama 3.1 70B	API key	Generous free tier	OpenAI-compatible, fast
Fireworks	Various	Free credits		OpenAI-compatible
Hugging Face Inference	Open-weight models	Token	Serverless free tier	Custom SDK or OpenAI-compatible via TGI
Anthropic	Claude family	API key	Paid; occasional research credits	Anthropic SDK

Provider	Notable Models	Access	Typical Free Rate	Integration
Ollama (local)	Qwen2.5-Coder 7B, DeepSeek-Coder-V2, Llama 3.2 3B, Phi-3, CodeGeeX4, Granite-Guardian	Local install	Unlimited, CPU-bound	OpenAI-compatible on localhost:11434
vLLM / llama.cpp (local, GPU) `[NEW]`	Any HF model	Local	Unlimited, GPU-bound	OpenAI-compatible

Because policies and limits change often, **all rate-limit specifics must be re-verified** when building the LiteLLM config (`[VERIFY-AT-INTEGRATION]`).

10.4 The AI Gateway — LiteLLM

All agents talk to a **single OpenAI-compatible endpoint**: `http://litellm:4000/v1`. LiteLLM handles:

- **Model routing:** logical name → concrete provider.
- **Failover:** `gemini-flash → groq-llama-3.3-70b → ollama/qwen2.5-coder:7b`.
- **Cost & budget:** per-virtual-key monthly budgets (`\$0` for free-tier agents).
- **Rate limiting:** honours per-provider RPM/RPD.
- **Prompt caching:** reuses identical prefixes across calls.
- **Audit logging:** every request/response stored (redacted) in Loki.
- **Hash-based deduplication:** identical recent calls return cached responses.
- **Streaming:** SSE passthrough for real-time agent UI feedback.

Reference `litellm_config.yaml`:

```yaml
model_list:
  - model_name: primary-code
    litellm_params:
      model: groq/llama-3.3-70b-versatile
      api_key: os.environ/GROQ_API_KEY
      rpm: 25
  - model_name: primary-code
    litellm_params:
      model: gemini/gemini-2.5-flash
      api_key: os.environ/GEMINI_API_KEY
      rpm: 5
  - model_name: primary-code
    litellm_params:
      model: openrouter/deepseek/deepseek-r1:free
      api_key: os.environ/OPENROUTER_API_KEY
      rpm: 20
  - model_name: primary-code
    litellm_params:
      model: ollama/qwen2.5-coder:7b
      api_base: http://ollama:11434

  - model_name: primary-general
    litellm_params:
      model: gemini/gemini-2.5-flash-lite
      api_key: os.environ/GEMINI_API_KEY
  - model_name: primary-general
    litellm_params:
      model: groq/llama-3.3-70b-versatile
      api_key: os.environ/GROQ_API_KEY
  - model_name: primary-general
    litellm_params:
      model: ollama/llama3.2:3b
      api_base: http://ollama:11434
```

```yaml
router_settings:
  routing_strategy: simple-shuffle
  fallbacks:
    - primary-code: ["primary-general"]
  retry_policy:
    BadRequestErrorRetries: 0
    AuthenticationErrorRetries: 0
    TimeoutErrorRetries: 3
    RateLimitErrorRetries: 5
  timeout: 30
  cache:
    type: redis
    host: redis
    port: 6379
    namespace: litellm
    ttl: 3600

litellm_settings:
  drop_params: true
  telemetry: false                  # no phone-home
  cache: true
  cache_params:
    type: redis
    similarity_threshold: 0.98
  set_verbose: false

general_settings:
  master_key: os.environ/LITELLM_MASTER_KEY
  database_url: os.environ/LITELLM_DB_URL
  store_model_in_db: true
  spend_logs_enabled: true
  alerting:
    - slack
  alerting_threshold: 1.0           # alert on any \$ spend (free-tier enforcement)
```

10.5 Local LLMs via Ollama

Run on the same A1.Flex VM (ARM-compatible builds). Models are quantised (Q4_K_M or Q5_K_M) for memory efficiency.

Model	Size (Q4)	Best Use	Approx. latency on A1.Flex CPU
`llama3.2:3b`	~2 GB	Log triage, summaries	2–5 s
`qwen2.5-coder:7b`	~4.5 GB	Code review, patch suggestions	15–40 s
`deepseek-coder-v2:16b`	~9 GB	Deep code analysis	30–90 s (too slow for interactive)
`phi3:mini`	~2.3 GB	Fast reasoning	3–7 s
`granite3-guardian:2b`	~1.5 GB	PII/secret detection	1–3 s
`nomic-embed-text`	~270 MB	Embeddings for RAG	ms-range
`bge-reranker-v2-m3`	~580 MB	Rerank RAG results	fast

When a GPU is available (even a modest one), **vLLM** or **llama.cpp with CUDA** delivers 10–30× the throughput. See §15.

10.6 RAG over Codebase `[NEW]`

A continuous indexing pipeline:

1. On every push, changed files are embedded via `nomic-embed-text` through Ollama.
2. Embeddings stored in **Qdrant** (free, self-hosted, Rust, extremely fast) or **ChromaDB**.
3. Agents use semantic retrieval before answering questions about the codebase — enabling "Why does our system do X?" queries from the dashboard.
4. **BM25 hybrid search** combines lexical and semantic retrieval (via `fastembed` + `tantivy`).
5. **Reranking** via `bge-reranker-v2-m3` improves precision.

10.7 Multi-Agent Orchestration `[NEW]`

For complex tasks (e.g., "refactor this module", "propose a fix for this bug"), agents compose via **LangGraph-style** DAGs:

- **Planner** (large model) → decomposes task into subtasks
- **Coder** (code model) → generates changes
- **Critic** (reasoning model) → reviews, finds issues
- **Tester** (code model) → generates tests, runs them in sandbox
- **Summarizer** (small model) → writes PR description

Written in Go (custom orchestrator) to avoid Python runtime on the worker node. Alt: Python `langgraph` service accessible via gRPC.

10.8 Safety Guardrails

- **Input/output classifiers** using **Granite-Guardian** (PII, toxicity, jailbreak attempts).
- **Constitutional constraints** encoded as system prompts and enforced via **nemo-guardrails** (rule-based).
- **Tool-use allowlist:** agents can only call pre-approved tools; other tool calls are rejected.
- **Budget breakers:** if spend > $0.00 (for free-tier agents), the agent is suspended.

10.9 Agent Communication Protocol

Internal agent messages flow through **NATS** (lighter than Kafka, runs in 20 MB RAM). Topics:

- `agent.request.{agent_id}`
- `agent.response.{agent_id}`
- `agent.event.{type}`
- `agent.broadcast`

Durable on disk for audit replay.

---

11. Observability — The Central Nervous System

11.1 Stack

All open-source; deployable via `dockprom` or a custom `docker-compose.observability.yml`:

- **Prometheus** — metrics, 30-day retention
- **Grafana** — dashboards, 10+ pre-built
- **Loki** — log aggregation, 7-day retention
- **Tempo** — distributed traces (OpenTelemetry)
- **Pyroscope** — continuous CPU & memory profiling
- **Alertmanager** — alert routing, deduplication, inhibition
- **Node Exporter** — host metrics
- **cAdvisor** — container metrics
- **Postgres Exporter** — DB metrics
- **Redis Exporter** — stream/memory metrics
- **Blackbox Exporter** — synthetic endpoint probing
- **Promtail** — log shipping to Loki
- **OpenTelemetry Collector** — trace/metric/log ingestion and routing
- **Parca** `[NEW]` — eBPF-based continuous profiler (zero-overhead)
- **Vector** `[NEW]` — alternative log pipeline with richer transforms

11.2 Metrics Catalog (Smart-Worker `/metrics`) `[RESOLVED]`

This resolves the inconsistency between `git_proxy_*` (Fixed v1) and `proxy_*` (Fixed v2). **Canonical prefix:** `gitpx_`.

Metric	Type	Labels	Purpose
`gitpx_push_success_total`	counter	`remote`, `repo`	Successful pushes
`gitpx_push_failure_total`	counter	`remote`, `repo`, `error_class`	Failed pushes
`gitpx_push_duration_seconds`	histogram	`remote`, `repo`	End-to-end push latency
`gitpx_push_bytes_total`	counter	`remote`, `repo`	Bytes transferred
`gitpx_queue_depth`	gauge	`remote`	Pending events per remote
`gitpx_queue_pending_seconds`	histogram	`remote`	Time in queue before processing
`gitpx_circuit_open`	gauge (0/1)	`remote`	Circuit breaker state
`gitpx_rate_limit_remaining`	gauge	`remote`	Last-observed upstream quota
`gitpx_lfs_object_transfer_total`	counter	`remote`, `status`	LFS object push count
`gitpx_lfs_bytes_total`	counter	`remote`	LFS bytes pushed
`gitpx_webhook_received_total`	counter	`type`, `repo`	Webhook events
`gitpx_webhook_rejected_total`	counter	`reason`	Rejected webhooks (HMAC fail, etc.)
`gitpx_secret_detection_total`	counter	`detector`, `severity`	Secret detections
`gitpx_upstream_drift_total`	gauge	`remote`, `repo`	Refs out of sync
`gitpx_attestation_total`	counter	`remote`, `status`	Sigstore attestations issued
`gitpx_token_days_until_expiry`	gauge	`remote`	Token expiry countdown
`gitpx_worker_processing_total`	gauge	`worker_id`	Active in-flight events per worker
`gitpx_plugin_invocations_total`	counter	`plugin`, `status`	WASM plugin calls
`gitpx_llm_request_total`	counter	`agent`, `provider`, `status`	LLM calls
`gitpx_llm_tokens_total`	counter	`agent`, `provider`, `direction`	Token usage
`gitpx_llm_cost_cents_total`	counter	`agent`, `provider`	Cost tracking (most == 0)
`gitpx_ratelimit_throttled_total`	counter	`remote`	Times throttle applied
`gitpx_backup_success_timestamp`	gauge	`target`	Unix ts of last successful backup
`gitpx_backup_size_bytes`	gauge	`target`	Latest backup size
`gitpx_disk_usage_ratio`	gauge	`mount`	Disk fullness
`gitpx_build_info`	gauge	`version`, `commit`, `goversion`	Build info sentinel

11.3 Alert Rules (Alertmanager → Telegram, Slack, Discord, Email, Matrix, PagerDuty)

```yaml
groups:
  - name: gitpx-critical
    rules:
      - alert: GiteaDown
        expr: up{job="gitea"} == 0
        for: 1m
        labels: { severity: critical }
        annotations:
          summary: "Gitea is down"
          runbook: "https://docs.gitpx.io/runbooks/gitea-down"

      - alert: SecretDetected
        expr: increase(gitpx_secret_detection_total[5m]) > 0
        labels: { severity: critical, page: "true" }
        annotations:
```

```
          summary: "Secret detected in pushed code"

      - alert: MirrorDrift
        expr: gitpx_upstream_drift_total > 0
        for: 30m
        labels: { severity: high }

      - alert: CircuitOpen
        expr: gitpx_circuit_open == 1
        for: 5m
        labels: { severity: high }

      - alert: HighMirrorFailureRate
        expr: |
          sum(rate(gitpx_push_failure_total[10m])) by (remote)
            /
          sum(rate(gitpx_push_success_total[10m]) + rate(gitpx_push_failure_total[10m])) by
(remote)
          > 0.1
        for: 10m
        labels: { severity: high }

      - alert: TokenExpiringSoon
        expr: gitpx_token_days_until_expiry < 14
        labels: { severity: medium }

      - alert: DiskPressure
        expr: gitpx_disk_usage_ratio > 0.85
        for: 10m
        labels: { severity: high }

      - alert: BackupStale
        expr: time() - gitpx_backup_success_timestamp > 7200
        labels: { severity: high }

      - alert: QueueStarvation
        expr: gitpx_queue_depth > 50
        for: 15m
        labels: { severity: medium }

      - alert: LLMProviderDown
        expr: |
          sum(rate(gitpx_llm_request_total{status="error"}[5m])) by (provider)
            /
          sum(rate(gitpx_llm_request_total[5m])) by (provider) > 0.5
        for: 5m
        labels: { severity: low }
```

11.4 Real-Time Dashboards (Grafana)

Pre-built dashboards shipped as `dashboards/*.json`:

- **Overview** — top-level health, pushes/min, success rate
- **Per-Upstream** — success/failure, latency, rate-limit remaining, queue depth
- **Per-Repo** — push volume, LFS transfer, secret detections
- **AI/LLM** — token usage, cost, failover chain execution

- **Security** — secret detections, CVE counts, OCI image signing status
- **Host** — CPU, memory, disk, network
- **Backup & DR** — backup success/failure, restore drill results
- **SLI/SLO** — user-facing reliability targets

11.5 Reports Generation `[NEW]`

Automated reports via **Grafana Reporting OSS alternative** (Grafana Image Renderer + custom Go templater):

- **Daily SLO report** — emailed to admins (HTML via `wkhtmltopdf` / `WeasyPrint`).
- **Weekly security digest** — CVEs, secrets caught, failed logins.
- **Monthly cost report** — LLM usage, storage growth, CI minutes consumed.
- **Quarterly DR drill report** — restore test results, runbook updates.
- **AI-generated narrative section** — Observability Interpreter agent writes a natural-language summary.

Output formats: HTML, PDF, Markdown, CSV, JSON, and `.ics` for scheduled deliveries.

---

12. Real-Time Events & Notification Framework `[NEW]`

12.1 Event Bus Architecture

All internal events (pushes, failures, security detections, user activity) flow through **NATS JetStream**. Separate from the **data-plane** Redis Streams (which handle the push-to-upstream pipeline), NATS carries the **control-plane / UX** events.

Why two buses? Redis Streams is optimised for at-least-once ordered durable delivery of a specific work item. NATS is optimised for fan-out to many subscribers (dashboards, mobile apps, chat bots) with filtering and low latency.

12.2 Event Taxonomy

```
gitpx.push.received                {repo, ref, before, after, pusher}
gitpx.push.scanned                 {repo, sha, result, findings}
gitpx.push.accepted                {repo, sha}
gitpx.push.rejected                {repo, sha, reason}
gitpx.upstream.push.started        {repo, upstream, sha}
gitpx.upstream.push.succeeded      {repo, upstream, sha, duration_ms, bytes}
gitpx.upstream.push.failed         {repo, upstream, sha, error, attempt}
gitpx.upstream.circuit.opened      {upstream}
gitpx.upstream.circuit.closed      {upstream}
gitpx.lfs.transfer.started         {repo, upstream, size}
gitpx.lfs.transfer.succeeded       {...}
gitpx.security.secret.detected     {repo, sha, detector, severity}
gitpx.security.cve.discovered      {repo, cve, severity}
gitpx.backup.started               {target}
gitpx.backup.succeeded             {target, size, duration}
gitpx.backup.failed                {target, error}
gitpx.dr.drill.started             {...}
gitpx.dr.drill.completed           {...}
gitpx.token.expiring               {upstream, days_left}
gitpx.agent.review.posted          {repo, pr, agent}
gitpx.agent.incident.triggered     {...}
gitpx.user.login                   {user, ip}
gitpx.user.pr.opened               {...}
gitpx.user.issue.opened            {...}
gitpx.system.config.reloaded       {scope}
gitpx.system.alert.fired           {name, severity}
```

12.3 Notification Integrations

Every event type can be routed to zero or more channels. Configured per-user, per-repo, per-severity.

Channel	Transport	Free Tier	Use Case
Email	SMTP (Mailgun/Brevo/Resend/Self-hosted Postfix)	300–3000/mo	Digest, audit, alerts
Telegram	Bot API (free)	Unlimited	Real-time alerts, 2-way commands
Slack	Incoming Webhooks + Slack App	Free workspaces	Team chat alerts
Discord	Webhooks + Bot	Free	Community repos
Matrix	Matrix protocol	Free (self-hosted)	FOSS-friendly alternative
Mattermost	Webhooks	Self-hosted free	Enterprise chat

Channel	Transport	Free Tier	Use Case
Microsoft Teams	Incoming Webhook	Free tier	Enterprise
SMS	Twilio free trial, TextBelt, MessageBird	Very limited free	Critical only
Push Notifications	Firebase Cloud Messaging, Apple Push	Free	Mobile apps
Web Push	VAPID / WebPush protocol	Free	Browser dashboard
RSS/Atom	Built-in feed endpoint	Free	Passive monitoring
Webhook (generic)	HTTP POST to custom URL	Free	Third-party integration
PagerDuty	API	Free tier exists	Critical on-call
Opsgenie	API	Free tier exists	Alt on-call
ntfy.sh `[NEW]`	Free public or self-hosted	Free	Simple push without accounts
Gotify `[NEW]`	Self-hosted	Free	Self-hosted push alternative
IRC `[NEW]`	IRC protocol	Free	Legacy/FOSS communities
Jabber/XMPP `[NEW]`	XMPP	Free	Legacy
Apprise `[NEW]`	Meta-library supporting 80+ services	Free	Single library, many destinations

The **Notification Service** (Go binary) subscribes to NATS, applies per-user filter rules (stored in Postgres), renders templates (using Go `text/template` + `sprig`), and dispatches. Template library ships with sensible defaults; users can override via the dashboard.

12.4 Two-Way Chat Bot Commands `[NEW]`

The Telegram/Slack/Discord bots support command interactions:

- `/status` — current health summary
- `/pushes 1h` — recent push log
- `/drift` — current drift status
- `/retry <upstream> <repo>` — force retry
- `/quarantine <upstream>` — pause an upstream
- `/reload` — trigger hot config reload
- `/review <pr_url>` — request an AI review
- `/explain <commit_sha>` — AI-generated commit summary
- `/runbook <alert>` — fetch runbook for an alert

Commands are authenticated via pre-registered user↔chat-ID mapping; authorized scopes configurable per user.

12.5 WebSocket / SSE Stream for UIs

The **Realtime Gateway** (Go, uses `nhooyr/websocket` and `r3labs/sse`) exposes:

- `wss://git.example.com/ws/events?topics=push.*,upstream.*` — filtered event stream
- `https://git.example.com/sse/events?topic=...` — SSE fallback

Used by the Web Dashboard, Desktop, Mobile apps to get live updates without polling. Scales to hundreds of concurrent clients on a single VM.

---

13. Language & Technology Matrix per Layer `[NEW]`

Every layer uses the **most proper tool** for the job:

Layer	Primary Language	Secondary	Rationale
Smart-Worker core	**Go**	—	Fast, concurrent, cross-compiled, `go-git`
Performance-critical paths (hashing, crypto, CUDA bindings)	**Rust**	C	Memory safety + predictable perf
GPU kernels	**CUDA C++**	PTX, Triton	Nvidia GPU acceleration

Layer	Primary Language	Secondary	Rationale
Plugin host	Go (wazero)	—	WASM runtime
Plugins (by community)	**Rust**, TinyGo, AssemblyScript, Zig, C	—	Any language compilable to WASM
Webhook Receiver	Go	—	Consistency with worker
Realtime Gateway	Go	—	WS/SSE
AI agent orchestrator	Go	Python (optional for LangGraph)	Low-dep binary preferred
RAG indexing service	Rust	Python	`fastembed-rs` + `tantivy`
Notification Service	Go	—	
Observability stack	Go (vendor-provided)	—	Prometheus/Loki/Tempo
Web dashboard	**TypeScript + React 19** or **Svelte 5**	—	Ecosystem, SSR via Next.js/SvelteKit
Web dashboard 3D/graph visualisations	TypeScript + **Three.js** / **D3.js** / **Sigma.js**	WebGL shaders (GLSL)	Fan-out graph, provenance trees
Desktop apps	**Rust + Tauri v2**	Optionally Wails (Go) or Flutter Desktop	Tiny binary, native perf, webview UI
Android app	**Kotlin + Jetpack Compose**	KMP shared code with iOS	Modern native
iOS app	**Swift 6 + SwiftUI**	KMP shared code	Modern native
HarmonyOS NEXT app	**ArkTS** (HarmonyOS SDK)	—	Required for pure-HarmonyOS
Aurora OS app	**Qt 6 + QML** (C++)	—	Aurora OS is Qt/QML based
Cross-platform mobile (alt)	**Flutter 3.x + Dart**	—	Single codebase for Android/iOS/HarmonyOS (preview)/desktop
Shared UI kit (optional)	**Kotlin Multiplatform + Compose Multiplatform**	—	Share UI code Android↔iOS↔Desktop
CLI (`gitpxctl`)	Go	—	Single binary everywhere
TUI (`gitpxtop`)	Go + **Bubbletea**	Rust + **ratatui**	Terminal dashboard
Browser extension (Chrome/Firefox/Edge)	TypeScript + Vite	—	MV3 manifest
VS Code extension	TypeScript	—	Required by VS Code ext API
JetBrains plugin	Kotlin	Java	Required by IDE ext API
Scripting / pipelines	**Bash (strict mode)** + **Just** / **Taskfile**	—	See §14 for the Bash SDK
Nix / build system	Nix	Bazel, Earthly	Reproducible builds
Infrastructure-as-code	**OpenTofu** (OSS fork of Terraform)	Pulumi	Cloud provisioning
Config management	Ansible	—	Host bootstrap
Container images	Distroless + multi-stage Dockerfiles	—	Minimal attack surface
Container runtime	**Podman** (rootless preferred)	Docker	Security-first
Orchestration (scale-out)	Kubernetes + **Helm**	Nomad, K3s	Standard
Service mesh	**Linkerd**	Istio	Simpler; lighter

Layer	Primary Language	Secondary	Rationale
Local dev	Docker Compose + **Devbox** (Nix-based)	—	Reproducible dev env

---

14. Bash Scripting SDK & Pipeline Framework `[NEW]`

14.1 Problem

Bash is ubiquitous but fragile. Most project bash is monolithic, untested, and unportable. gitpx ships a disciplined Bash SDK to make pipeline scripts reliable.

14.2 `gitpx-bash-sdk` Structure

```
tools/bash-sdk/
  lib/
    bootstrap.sh        # strict mode, traps, colours, cleanup
    log.sh              # structured JSON logging to stdout/file
    retry.sh            # exponential backoff with jitter
    assert.sh           # assertion helpers for scripts + tests
    tempdir.sh          # safe mktemp with auto-cleanup traps
    lock.sh             # flock-based mutual exclusion
    json.sh             # jq wrappers
    http.sh             # curl wrappers with timeouts, retries
    gitops.sh           # safe wrappers around git commands
    notify.sh           # send to Telegram/Slack/Discord
    secrets.sh          # load env vars from sops-encrypted file
    metrics.sh          # push custom metrics to Prometheus Pushgateway
    validate.sh         # shellcheck + shfmt integration
  bin/
    gitpx-backup        # executable scripts built on the lib
    gitpx-restore
    gitpx-health
    gitpx-rotate-tokens
    gitpx-drift-check
    gitpx-emergency-purge
  tests/
    bats/
      lib.bats          # BATS tests for every lib function
      bin.bats
  Taskfile.yml          # all entry points
```

14.3 Bootstrap Idiom

Every script starts with:

```bash
#!/usr/bin/env bash
# shellcheck shell=bash
set -euo pipefail
IFS=\$'\n\t'
source "\$(dirname "\${BASH_SOURCE[0]}")/../lib/bootstrap.sh"
gitpx::bootstrap "\$@"
trap 'gitpx::on_exit \$?' EXIT
```

`gitpx::bootstrap` sets up:

- Strict mode (`-euo pipefail`, `IFS`)
- `shopt -s inherit_errexit` (Bash 4.4+)
- Error trap with stack-trace printer
- Coloured + structured logging
- Temp dir with auto-cleanup
- Signal handlers (INT/TERM → graceful drain)
- Version detection + dependency check

14.4 Task Runner

Prefer **Just** or **Taskfile** (YAML-based) over Makefiles — clearer, better portability.

```yaml
# Taskfile.yml
version: '3'
tasks:
  lint:
    cmds:
      - shellcheck --enable=all tools/bash-sdk/{lib,bin}/**/*.sh
      - shfmt -d -s -i 2 tools/bash-sdk/
  test:
    cmds:
      - bats tools/bash-sdk/tests/bats/
  coverage:
    cmds:
      - kcov --include-path=tools/bash-sdk/lib coverage bats tools/bash-sdk/tests/bats/
```

14.5 Reusable Pipeline Patterns

- `retry::with_backoff` — wrap any command with exponential retry
- `http::post_json` — safe JSON POST with TLS, timeout, status handling
- `gitops::safe_push` — implements §4.4 policy in bash
- `notify::critical "message"` — multi-channel critical alert
- `metrics::gauge <name> <value>` — push gauge to Pushgateway
- `secrets::load_sops <file>` — decrypt SOPS file into env

14.6 Testing

- `bats-core` — test framework (TAP output)
- `shellcheck` — static analysis (treat warnings as errors in CI)
- `shfmt` — formatter
- `kcov` — coverage (target ≥ 90% on lib/)
- **Property-based tests** via `bats-assert` + shell generators for critical retry/backoff code

14.7 Portability

- POSIX-sh fallback for bootstrap (GNU Bash not always available on Aurora/HarmonyOS shell environments).
- macOS BSD utils compatibility layer (e.g., `gsed` detection).
- Busybox environments (tested in distroless images).

---

15. GPU / CUDA Acceleration Layer `[NEW]`

Optional but game-changing when a GPU is available (self-hosted with NVIDIA GPU, or paid cloud with minimum spend).

15.1 Use Cases

Workload	CPU baseline	GPU speedup	Library
Local LLM inference (7B model)	20–40 s / response	0.5–2 s / response	**vLLM**, **llama.cpp (CUDA)**, **TensorRT-LLM**
Embedding generation (nomic, bge)	~50 docs/s	~2,000 docs/s	**fastembed** (ONNX Runtime CUDA), **CTranslate2**
Code similarity / clone detection	Slow n^2	GPU tensor ops	**FAISS GPU**, **ScaNN**
Rolling-hash fingerprinting for LFS dedup	Gigabit CPU	10×+	Custom CUDA kernel or `libhashcat` primitives
SHA-256 / BLAKE3 bulk hashing	CPU-bound	GPU parallel	CUDA SHA, `blake3-cuda`
Regex / SIMD scanning (e.g., secret scanning on big pushes)	Hyperscan CPU	Hyperscan uses AVX; GPU via `cuRE`	
Video / visual-regression diff	CPU OpenCV	OpenCV CUDA	`cv::cuda::*`

Workload	CPU baseline	GPU speedup	Library
Image OCR for issue attachments	Tesseract CPU	PaddleOCR GPU	PaddleOCR, EasyOCR

15.2 Runtime Stack

- **NVIDIA Container Toolkit** — makes GPU available to Docker/Podman.
- **CUDA 12.x** minimum; **cuDNN 9.x** for deep learning workloads.
- **NCCL** if multi-GPU.
- **TensorRT-LLM** for production LLM serving.
- **Triton Inference Server** for multi-model serving.

15.3 OpenCV Integration `[NEW]`

Where OpenCV adds real value in a Git proxy:

1. **Visual regression testing for docs sites** — screenshot diffs (§7.13) using SSIM, perceptual hashing.
2. **PR screenshot attachment OCR** — extract text from uploaded images for indexing/search.
3. **UI testing** of the Web Dashboard itself — Playwright + OpenCV for visual assertions.
4. **Automated UI mockup extraction** — from design files attached to issues.
5. **CAPTCHA bypass detection on auth pages** — anomaly detection of bot-like patterns.
6. **QR code generation/scanning** — for mobile app pairing with the proxy (scan QR → configure API token).
7. **Plot rendering** — generate PNG dashboards from metrics in scripts.

The worker does not need OpenCV; the **QA pipeline** and **Realtime Gateway** do. Runs as a separate `gitpx-cv` microservice (Python + OpenCV-CUDA when GPU present, otherwise CPU).

15.4 eBPF Observability `[NEW]`

Use `cilium/ebpf` to attach kernel probes to:

- `sys_enter_openat` — detect unexpected file access.
- `tcp_connect` — track egress to each upstream with zero overhead.
- `sched_process_exec` — audit which processes the worker forks.
- `net_dev_xmit` — per-interface bandwidth.

Surfaced as Prometheus metrics + flame graphs in **Pyroscope**.

---

16. Containerization & Virtualization

16.1 Container Strategy

- **Multi-stage Dockerfiles** for all services. Final stage is **distroless** (`gcr.io/distroless/static:nonroot` for Go, `gcr.io/distroless/cc:nonroot` for Rust/C).
- **Images signed** with `cosign sign --yes ghcr.io/gitpx/<svc>@sha256:....`
- **SBOMs attached** to images via `cosign attach sbom`.
- **Rootless container execution** (UID 65532 nonroot).
- **ReadOnlyRootFilesystem: true** in K8s deployments.
- **Seccomp profiles** — custom profile in `deploy/seccomp/gitpx.json`.
- **AppArmor profiles** for host-level confinement.
- **Image scanning** via Trivy + Grype in CI; image published only if zero High/Critical CVEs or waivers documented.
- **Buildkit** with cache mounts for speed.
- **Multi-arch builds** for `amd64`, `arm64`, `riscv64` via `docker buildx`.

16.2 Alternative: Podman + Quadlet

For Fedora/RHEL-based hosts, Podman with **Quadlet** (systemd-native container units) provides a simpler, systemd-native orchestration than Docker Compose.

16.3 KVM / QEMU / Firecracker `[NEW]`

For defense-in-depth isolation — especially when running **untrusted code** (AI-generated patches being tested, PR test runners, fuzzer payloads):

- **Firecracker** microVMs for ultra-fast (< 125 ms boot), lightweight isolation. Used by the **sandbox test runner** that executes AI-proposed patches against a test suite without giving them access to the host.

- **Kata Containers** for container workloads requiring VM-level isolation (when running in K8s).
- **QEMU/KVM** for full VMs, e.g., running a test instance of Windows Server for cross-platform CI.
- **LXD/Incus** for system containers (full OS, shared kernel, snapshots).

Use cases:

- Running AI-generated patches in a Firecracker VM with no network, ephemeral filesystem, 2 min wallclock timeout.
- Running chaos-engineering targets in disposable microVMs.
- Running CI test matrices for multiple OS targets.

16.4 Orchestration

Phase 1 (current): Docker Compose on single VM.

Phase 2 (growth): **K3s** (lightweight Kubernetes) on a 3-node cluster.

Phase 3 (scale): Full Kubernetes with **Helm** charts. Supporting tech:

- **ArgoCD** for GitOps
- **Linkerd** for service mesh (preferred over Istio for simplicity)
- **cert-manager** for TLS
- **External-DNS** for automatic DNS
- **KEDA** for event-driven autoscaling (scale workers based on Redis queue depth)
- **Strimzi** for Kafka (if/when moving from Redis Streams to Kafka)

---

17. Modern Build Systems & Reproducibility `[NEW]`

17.1 Build System Strategy

Tool	Scope	Why
Nix / Flakes	System-level reproducibility	Pure, reproducible, content-addressed
Bazel (with rules_go, rules_rust, rules_nodejs)	Large monorepo with deep graphs	Hermetic, incremental, remote cache
Buck2 `[NEW]`	Alternative to Bazel, faster for many use cases	Facebook/Meta rewrite
Earthly	CI-friendly reproducible pipelines	Dockerfile-like syntax for repeatable builds
Turborepo / **Nx**	Frontend + CLI monorepo	JS/TS-oriented, caching-aware
Cargo workspaces	Rust monorepo	Native Rust
Go workspaces (`go.work`)	Go monorepo	Native Go
Gradle with **Version Catalogs**	Android + KMP	Native Android
Xcode + Swift Package Manager	iOS	Native iOS
OHPM / Hvigor	HarmonyOS NEXT	Required by HarmonyOS
qmake / CMake	Aurora OS (Qt)	Required by Aurora

Chosen architecture:

- Monorepo layout managed by **Nix flakes** at the top level.
- **Bazel** for Go/Rust/Proto polyglot core (smart-worker, receiver, gateway, agents).
- **Turborepo** for the frontend apps + TypeScript shared libraries.
- **Per-platform native tools** for mobile (Gradle, SwiftPM, Hvigor, Qt).
- **Earthly** for end-to-end CI pipelines.

17.2 Reproducible Builds & SLSA

- Builds are **hermetic** — no network access during compile.
- Source dates are pinned (`SOURCE_DATE_EPOCH`).
- **SLSA Level 3** provenance attached to every release artefact.
- **Verifier** tool (`gitpx verify-build`) confirms a given binary matches its published provenance.

17.3 Dependency Management

- **Renovate Bot** — automated dependency PRs.
- **Grouped updates** to reduce PR noise.
- **Security-only auto-merge** with full CI green.
- **Manual review** for major version bumps.

---

18. Testing Strategy — 100% Coverage Blueprint

18.1 Testing Pyramid

```
        / \
       /E2E\              — ~50 scenarios; slow; run on every PR + nightly
      /-----\
     / UI/CV \            — Visual regression (OpenCV); nightly
    /---------\
   /Integration\          — Testcontainers; per-PR
  /-------------\
 /  Unit Tests   \        — Fast, extensive; per-commit
/-----------------\
```

18.2 Test Types Matrix

Test Type	Target	Tools	Coverage Goal	CI Frequency
Unit (Go)	Worker, Receiver, Gateway	`go test`, `testify`, `gomock`	**100% statements, ≥ 90% branches** on `internal/`	Every commit
Unit (Rust)	Performance-critical crates	`cargo test`, `proptest`	100% + property-based	Every commit
Unit (TS)	Web frontend	**Vitest**, `testing-library/react`	≥ 90%	Every commit
Unit (mobile)	Android	JUnit 5, Kotest, Turbine	≥ 85%	Every commit
Unit (mobile)	iOS	XCTest, Quick/Nimble	≥ 85%	Every commit
Unit (mobile)	HarmonyOS	ArkTS test framework	≥ 85%	Every commit
Unit (mobile)	Aurora	Qt Test	≥ 85%	Every commit
Unit (Bash)	Bash SDK	`bats-core`, `shellcheck`, `kcov`	≥ 90% on lib	Every commit
Integration	Worker ↔ Gitea ↔ Redis	`Testcontainers-go` (Gitea + Postgres + Redis containers)	All critical paths	Every PR
Integration (adapter)	Each provider adapter	Testcontainers + mock HTTP servers (WireMock)	All response code classes	Every PR
Contract	Upstream API contracts	`Pact`	Per-provider contract	Nightly
E2E (protocol)	`git push` over HTTPS/SSH	Bash + real git client	Happy path + LFS + force-push-block	Every PR
E2E (UI)	Web dashboard	**Playwright** (multi-browser)	Critical flows	Every PR
E2E (mobile)	Android	Espresso + UI Automator	Critical flows	Nightly
E2E (mobile)	iOS	XCUITest	Critical flows	Nightly
Visual Regression	Docs, dashboard	Playwright + OpenCV SSIM / `pixelmatch`	Key pages	Nightly
Chaos	Whole system	**LitmusChaos** / **Chaos Mesh**	Pod kill, latency, partition	Weekly

Test Type	Target	Tools	Coverage Goal	CI Frequency
Fuzz	Parsers (config, webhook JSON, Git refs)	`go-fuzz`, Rust `cargo-fuzz`, OSS-Fuzz	Continuous	Continuous
Property-based	Core logic (backoff, ref-mapping, retry)	`rapid` (Go), `proptest` (Rust), `fast-check` (TS)	Invariants hold	Every PR
Mutation	Go code	`go-mutesting`, `gremlins`	≥ 75% mutation score	Weekly
Load / Stress	All services	**k6**, Vegeta, Locust	Sustain 50 RPS pushes	Pre-release
Soak	System under continuous load	k6 + chaos	72 h continuous	Pre-release
Security (SAST)	Source	CodeQL, Semgrep, `gosec`, `clippy --all`	Zero High/Critical findings	Every PR
Security (DAST)	Running service	OWASP ZAP, **Nuclei**	Zero High/Critical	Nightly
Security (Secrets)	Push diffs	Gitleaks, TruffleHog, detect-secrets	Zero escapes	Every push
SCA (Deps)	Lockfiles	Trivy fs, Grype, `govulncheck`, `cargo audit`, `npm audit --production`	Zero High/Critical	Every PR
Container scan	OCI images	Trivy, Grype, Dockle	Zero High/Critical + best-practice pass	Every build
IaC scan	Dockerfiles, K8s manifests, Terraform	Checkov, KICS, tfsec	Zero High/Critical	Every PR
Compliance	Licenses	OSS Review Toolkit (ORT), `license-finder`	No forbidden licenses	Every PR
Accessibility	Web/mobile UIs	axe-core, Lighthouse, iOS Accessibility Inspector, Android Accessibility Scanner	WCAG 2.2 AA	Nightly
Performance budgets	Web	Lighthouse CI	Budget file enforced	Every PR
Disaster Recovery	Backup → restore	Custom GitHub Action that provisions a fresh VM	100% data restored, RTO ≤ 60 min	Monthly
Penetration testing	Whole system	Manual + ZAP + custom	Report quarterly	Quarterly

18.3 Test Infrastructure

- **GitHub Actions** workflows in `.github/workflows/*.yml` — 2,000 free minutes/month, plus optional self-hosted runner on the OCI VM for unlimited runs of heavy tests (chaos, soak).
- **Gitea Actions** as redundant CI — runs identical workflows on the proxy itself, satisfying dog-fooding + independence.
- **Buildjet / WarpBuild** free tiers considered for faster arm64 builds.
- **Testcontainers** for integration tests — spins up real Postgres, Redis, Gitea, LiteLLM in Docker.
- **k6 Cloud** free tier for distributed load tests (alternative: self-hosted k6 on OCI VM).
- **Sentry** (free dev plan) for runtime error telemetry from clients.

18.4 Test Data & Fixtures

- **Fake Git repos** generated by a fixture library with: small/medium/large repos, LFS objects, submodules, long histories (10^4 commits), wide branch sets, binary files, Unicode pathnames, deeply nested paths.
- **Synthetic webhook payloads** — captured real payloads from each provider, anonymised.
- **Mock upstream servers** — WireMock-style stubs that simulate GitHub/GitLab/Gitee APIs including rate limits, 5xx, slow responses, partial failures.
- **Chaos library** — canned “evil” inputs: malformed packs, corrupted deltas, unicode confusables in ref names.

18.5 Definition of Done for Tests

A feature is **done** when:

- Unit tests written and pass.
- Integration test for the feature in the larger system.
- At least one E2E scenario exercises the feature end-to-end.

- Metrics exposed + at least one Grafana panel + alert rule.
- Documentation updated.
- Threat model reviewed for the feature.
- CHANGELOG entry.

---

19. Client Applications Ecosystem `[NEW]`

A system of this scope deserves a full suite of clients. Every client is optional — the proxy functions with only vanilla `git`. But we provide delightful clients for operators, maintainers, and developers.

19.1 Web Dashboard

Stack: SvelteKit (Svelte 5) + TypeScript + TailwindCSS + ShadCN-Svelte. Alternative: Next.js (React 19 RSC).

Rationale for Svelte 5: Smallest bundle, reactive runes, low cognitive overhead, excellent for real-time dashboards. Next.js remains a defensible alternative if the team prefers React.

Key screens:

- **Overview** — live health map, pushes/min, success rate.
- **Repo Detail** — upstream map with per-upstream state (green/amber/red), last-push timeline.
- **Push Live View** — real-time fan-out animation as a push happens (WebSocket stream).
- **Provenance Explorer** — browse the Rekor-attested commits, verify signatures in-browser.
- **Drift Dashboard** — per-repo divergence heatmap.
- **AI Reviews** — recent PR reviews, accept/dismiss feedback trainer.
- **Security Center** — secrets caught, CVEs, licence issues, SBOMs.
- **Backup & DR** — last backup, restore button (with confirm dialog), drill history.
- **Notifications** — per-user subscription management.
- **Settings** — upstreams config editor (YAML validated live, “Apply” button triggers hot-reload).
- **Audit Log** — every config change, every manual action.
- **Terminal** — in-browser xterm.js gitpxctl shell (WebSocket → SSH → wrapped gitpxctl).

Design system:

- **Design tokens** (CSS custom properties) for light/dark, high-contrast, reduced-motion.
- **Keyboard-first navigation** — every action is accessible via shortcut.
- **Command palette** (⌘K / Ctrl+K).
- **Accessibility** — WCAG 2.2 AA; screen-reader tested.
- **i18n** — English, Serbian, Russian, Chinese (Simplified + Traditional), Japanese, Korean, Arabic (RTL), Spanish, French, German, Portuguese.

19.2 Desktop Apps (Linux / Windows / macOS)

Preferred stack: Tauri v2 (Rust host + webview UI shared with the web dashboard).

Why Tauri over Electron:

- 10–30× smaller binary (~10 MB vs 100+ MB).
- Lower memory (~30 MB vs 300+ MB).
- Uses OS webview (WebKit on macOS, WebView2 on Windows, WebKitGTK on Linux) — no bundled Chromium.
- Native system access in Rust (no fragile Node bindings).
- Built-in updater, signed updates, sidecar process management.

Alt stacks documented for contingency:

- **Wails v2** (Go host + webview) — if the team prefers Go across the board.
- **Flutter Desktop** — if Flutter is chosen for mobile (§19.3 alt).
- **Compose Multiplatform Desktop** (Kotlin) — if KMP is the cross-platform strategy.

Distribution:

- Linux: `.deb`, `.rpm`, **AppImage**, **Flatpak**, **Snap**.
- Windows: MSIX + NSIS installer + portable `.exe`.
- macOS: Notarized `.dmg` + `.pkg`, and Homebrew cask.
- All auto-update via Tauri updater with cosign signature verification.

Features beyond the web app:

- OS notifications (via `tao`).
- System tray with push status.
- Deep OS integration: “Open in IDE” buttons, URL protocol handler `gitpx://`.
- Offline queue — captures actions while disconnected, syncs on reconnect.
- Local repo-watcher — warns of push issues before they happen.

19.3 Mobile Apps

Android — Kotlin + Jetpack Compose

- **Minimum SDK:** API 26 (Android 8.0), covering ~94% of devices.
- **Architecture:** MVVM + Clean Architecture; UDF via Compose State.
- **Libraries:** Jetpack Compose, Navigation 3, Kotlin Coroutines + Flow, Hilt DI, Ktor client, SQLDelight (local), DataStore, WorkManager, Jetpack Glance (home-screen widget showing push status!).
- **Features:** Push notifications via FCM, biometric unlock, dark/light theme, Material You dynamic colour, tablet + foldable support.

iOS — Swift 6 + SwiftUI

- **Minimum iOS:** 17.
- **Architecture:** SwiftUI + Observation macro + async/await + Swift Concurrency.
- **Libraries:** SwiftData or GRDB for persistence, Alamofire or URLSession, Swift Charts.
- **Features:** APNs push, Face ID / Touch ID, Dynamic Island live activity showing push progress (iPhone 14 Pro+), iPad split view, macOS Catalyst option.

HarmonyOS NEXT — ArkTS

- **Language:** ArkTS (TypeScript-based, HarmonyOS-specific).
- **Framework:** ArkUI declarative.
- **Distribution:** AppGallery.
- **Features:** Harmony distributed capabilities (cross-device continuity — start reviewing a PR on phone, continue on watch/tablet), stylus support on PaperMatte devices.

Aurora OS — Qt 6 + QML (C++)

- **Reason:** Aurora OS is Qt-based. Native apps use Qt/QML; there is no WebView-based alternative that would feel native.
- **Framework:** Qt 6, QML, Aurora-specific APIs for notifications, security zones.
- **Distribution:** Aurora Store.

Cross-platform alternative — Flutter 3.x + Dart

For teams that prefer one codebase over native best-in-class:

- **Covers:** Android, iOS, Web, Linux/Win/macOS Desktop, and **HarmonyOS NEXT** via **OpenHarmony Flutter port** (`[VERIFY-AT-INTEGRATION]` — actively developed, preview-quality).
- Does **not** cover Aurora OS — Aurora is Qt/QML only.

Cross-platform alternative — Kotlin Multiplatform (KMP) + Compose Multiplatform

- Shared UI for Android + iOS + Desktop (experimental).
- Native platform-specific code where needed.
- No HarmonyOS or Aurora support in KMP’s core; would still need ArkTS/Qt for those.

Recommendation: Native apps per platform for best UX, with a shared **Rust core library** (`gitpx-core`) compiled via:

- `.aar` for Android (via `cargo-ndk`).
- `.xcframework` for iOS.
- `.so`/`.dll`/`.dylib` for Desktop.
- `.node` addon for Electron/Tauri if needed.

The Rust core handles: API calls, webhook signature verification, local encryption, model inference (via `candle`). UIs are pure-native for best-in-class feel.

19.4 CLI — `gitpxctl`

Go binary, single static build, works everywhere `git` does.

```
gitpxctl login                      # interactive OAuth flow
gitpxctl status                     # overall health
gitpxctl repo list
gitpxctl repo show acme/foo         # fan-out map
gitpxctl upstream add github https://github.com/acme/foo.git
gitpxctl upstream disable gitlab
gitpxctl upstream test gitee        # dry-run auth check
gitpxctl drift                      # drift across all upstreams
gitpxctl drift resolve acme/foo github
gitpxctl reload                     # hot-reload config
gitpxctl backup now
gitpxctl backup list
gitpxctl restore --to-vm user@host
gitpxctl attest verify acme/foo abc1234  # Rekor signature verification
gitpxctl agent review 123
gitpxctl dr drill --dry-run
gitpxctl tokens expiring
gitpxctl watch                      # live event stream in terminal
```

Supports `--output json|yaml|table` and `--watch` for streaming.

19.5 TUI — `gitpxtop`

Terminal UI inspired by `k9s`. Go + **Bubbletea** + **Lip Gloss**. Alternative Rust implementation with **ratatui**.

Live view of:

- Push events (streaming)
- Queue depths per upstream
- Circuit breaker state map
- Latency heatmap
- Log tail with filtering

19.6 Browser Extensions (Chrome, Firefox, Edge)

Manifest V3, TypeScript + Vite.

Features:

- Badge shows unread AI review comments.
- Right-click a GitHub PR → "Open equivalent in gitpx".
- "Sync now" button on any Git web UI (context menu).
- Inline inline-comment bridge: AI review comments on GitHub pages rendered from gitpx.
- Repo-sync indicator shown next to repo title (green/amber/red).

19.7 IDE Extensions

- **VS Code** — TS extension, shows drift/attestation in status bar, "gitpx: push & verify" command.
- **JetBrains** (IntelliJ/WebStorm/etc.) — Kotlin plugin, same features.
- **Neovim** — Lua plugin `gitpx.nvim`.
- **Emacs** — `gitpx.el`.

---

20. Development Phases — Fine-Grained Task Breakdown

Phases, sub-phases (milestones), tasks (stories), and sub-tasks (micro-tasks) for project management.

Every task has:

- **ID:** `T-{phase}.{sub}.{task}`
- **Estimated effort:** `XS` (≤ 1 h), `S` (≤ 0.5 d), `M` (≤ 2 d), `L` (≤ 5 d), `XL` (≤ 10 d).
- **Dependencies:** `[T-a.b.c]`
- **Deliverables:** Verifiable output.

Phase 0 — Pre-Development Setup `[NEW]`

Goal: establish project governance, tooling, standards.

Sub-phase 0.1 — Governance

- T-0.1.1 (S) — Draft `CODE_OF_CONDUCT.md`, `CONTRIBUTING.md`, `SECURITY.md`, `GOVERNANCE.md`.
- T-0.1.2 (S) — Choose license (Apache-2.0 suggested); add `LICENSE`, SPDX headers on all files.
- T-0.1.3 (XS) — Register `gitpx.io` domain (or settle on final name).

- T-0.1.4 (S) — Create GitHub org, OCI tenancy, Backblaze account, Cloudflare account.
- T-0.1.5 (S) — Draft `ADR-0001`: architectural decision record for chosen architecture (§4.2).

Sub-phase 0.2 — Tooling

- T-0.2.1 (M) — Set up **Nix flake** for reproducible dev env.
- T-0.2.2 (S) — Scaffold monorepo with Bazel + Turborepo + per-platform native tools.
- T-0.2.3 (S) — Pre-commit hooks (`gitleaks`, `shellcheck`, `shfmt`, `go vet`, `clippy`, `prettier`, `eslint`).
- T-0.2.4 (S) — Renovate config.
- T-0.2.5 (S) — GitHub Actions workflow templates: lint, test, build, release.
- T-0.2.6 (XS) — Slack/Discord/Telegram channels for alerts.

Phase 1 — Infrastructure Provisioning & Zero-Trust Networking

Sub-phase 1.1 — Compute

- T-1.1.1 (S) — Register OCI Always Free; provision `VM.Standard.A1.Flex` (Ubuntu 22.04 LTS ARM64), 4 OCPU, 24 GB RAM, 200 GB block vol.
- T-1.1.2 (XS) — SSH key-only authentication; disable password auth.
- T-1.1.3 (XS) — `ufw` / iptables: default DENY ingress except loopback and cloudflared.
- T-1.1.4 (S) — Install Docker CE + Compose v2; verify rootless operation.
- T-1.1.5 (S) — Install Podman + Quadlet as alternative (documented).
- T-1.1.6 (S) — Provision secondary compute (GCP e2-micro) for geo-redundancy (optional).

Sub-phase 1.2 — Networking

- T-1.2.1 (S) — Create Cloudflare account; add domain; configure DNS.
- T-1.2.2 (M) — Deploy `cloudflared` as Docker container; create tunnel; route `git.domain → localhost:3000`.
- T-1.2.3 (S) — Enable Cloudflare WAF, Bot Fight Mode, Under Attack default rules.
- T-1.2.4 (S) — Verify zero open ingress ports with external nmap scan.
- T-1.2.5 (S) — Configure Cloudflare Access for admin endpoints (`/admin`).
- T-1.2.6 (S) — Set up Tailscale as OOB admin backup path.
- T-1.2.7 (S) — DoH (`dnscrypt-proxy`) on host.
- T-1.2.8 (XS) — `chrony` NTP with multi-source configuration.

Sub-phase 1.3 — Observability Backbone

- T-1.3.1 (M) — Deploy `dockprom` stack.
- T-1.3.2 (S) — Configure Prometheus scrape targets.
- T-1.3.3 (S) — Provision Grafana org, dashboards folder, data sources.
- T-1.3.4 (S) — Configure Loki + Promtail.
- T-1.3.5 (S) — Configure Alertmanager with Telegram, Slack, Discord, Matrix, Email receivers.

Phase 2 — Core Git Forge Deployment

Sub-phase 2.1 — Gitea Core

- T-2.1.1 (S) — Author `docker-compose.gitea.yml` (Gitea + Postgres + Redis + Caddy optional).
- T-2.1.2 (S) — Deploy; complete web installer; create admin account.
- T-2.1.3 (S) — Create first organization `gitpx-test`.
- T-2.1.4 (S) — Configure `app.ini` hardening from §4.1.
- T-2.1.5 (S) — Enable `/metrics` endpoint with token.
- T-2.1.6 (S) — Configure OAuth2/OIDC providers (GitHub, Google).
- T-2.1.7 (S) — Test LFS with a sample ≥ 50 MB file.

Sub-phase 2.2 — Branch Protection & Policy

- T-2.2.1 (S) — Draft default branch-protection template (§4.1).
- T-2.2.2 (M) — Install Gitea server-side pre-receive hook (`gitleaks protect`) per §9.4.
- T-2.2.3 (S) — Verify force-push blocked on protected branches.
- T-2.2.4 (S) — Configure 2FA enforcement for admins.

Sub-phase 2.3 — Backup Foundations

- T-2.3.1 (S) — Provision Backblaze B2 bucket + Application Key (scoped).
- T-2.3.2 (S) — Enable Object Lock (compliance mode, 30 d retention).
- T-2.3.3 (M) — `tools/bash-sdk/bin/gitpx-backup` writing `gitea dump` + Postgres + volume to Restic → B2.
- T-2.3.4 (S) — Cron: hourly Restic, daily Gitea dump.
- T-2.3.5 (S) — Verify restore in test VM (end-to-end).

Phase 3 — Event Bus & Smart-Worker Development (Core Engine)

Sub-phase 3.1 — Redis & Webhook Receiver

- T-3.1.1 (S) — Deploy Redis 7 container with `--appendonly yes`, persistence volume.
- T-3.1.2 (S) — Configure Redis ACL for per-service users.
- T-3.1.3 (M) — Scaffold Go Webhook Receiver; implement `POST /webhook` with HMAC verification.
- T-3.1.4 (M) — Integrate Gitleaks scanner into receiver (blocking gate).
- T-3.1.5 (S) — Receiver writes event to per-repo-per-upstream Redis Streams.
- T-3.1.6 (S) — Configure Gitea org webhook → receiver URL.
- T-3.1.7 (S) — Unit + integration tests (Testcontainers).

Sub-phase 3.2 — Smart-Worker Core

- T-3.2.1 (L) — Scaffold Go module; layout packages per §5.2.
- T-3.2.2 (M) — Implement `config` package: load, validate (JSON Schema), hot-reload via `fsnotify` + Redis Pub/Sub.
- T-3.2.3 (M) — Implement `events` package: ULID-keyed GitEvent marshalling.
- T-3.2.4 (M) — Implement `redis` package: XREADGROUP consumer, XPENDING reaper, idempotency locks.
- T-3.2.5 (L) — Implement `gitops` package: ephemeral clone, safe push refspec logic (§4.4), `go-git` wrappers.
- T-3.2.6 (L) — Implement `adapters/generic` with Universal Git Adapter.
- T-3.2.7 (M) — Implement `adapters/github` with rate-limit header parsing + repo auto-create.
- T-3.2.8 (M) — Implement `adapters/gitlab`.
- T-3.2.9 (M) — Implement `adapters/gitee`.
- T-3.2.10 (M) — Implement `adapters/bitbucket`.
- T-3.2.11 (M) — Implement `adapters/gitflic`.
- T-3.2.12 (M) — Implement `adapters/gitverse`.
- T-3.2.13 (M) — Implement `adapters/codeberg` (+ `forgejo`, `sourcehut`).
- T-3.2.14 (M) — Implement `ratelimit` package (token bucket + header parsing + adaptive).
- T-3.2.15 (M) — Implement `lfs` package with Dragonfly integration.
- T-3.2.16 (M) — Implement `attest` package: Sigstore/Cosign/Rekor.
- T-3.2.17 (M) — Implement `secrets` package: env vars only, zeroize on exit, no stdout.
- T-3.2.18 (S) — Implement `health` endpoints `/healthz`, `/readyz`, `/diag`.
- T-3.2.19 (M) — Implement `metrics` package with every metric in §11.2.
- T-3.2.20 (M) — Implement `plugin` package with wazero WASM host.
- T-3.2.21 (M) — Implement circuit breaker (`gobreaker`).
- T-3.2.22 (M) — Implement exponential backoff with jitter (`go-retry`).
- T-3.2.23 (S) — Implement graceful shutdown (SIGTERM drain).
- T-3.2.24 (L) — Unit tests — **100% statement, ≥ 90% branch coverage**.
- T-3.2.25 (L) — Integration tests with Testcontainers (Gitea + Postgres + Redis + mock upstreams).
- T-3.2.26 (S) — Multi-arch Dockerfile + build workflow.
- T-3.2.27 (XS) — Deploy Worker container; wire to Gitea + Redis.

Sub-phase 3.3 — Drift Detector

- T-3.3.1 (M) — `gitpx-drift-check` bash script using `git ls-remote`.
- T-3.3.2 (S) — Inject resync events when drift detected.
- T-3.3.3 (S) — Grafana dashboard for drift.

Phase 4 — Security, Provenance & Compliance

Sub-phase 4.1 — Secret Annihilation

- T-4.1.1 (S) — Ship pre-commit config template in repo init.
- T-4.1.2 (S) — Server-side pre-receive hook integrated.
- T-4.1.3 (M) — Secret detection webhook flow fully validated (unit + integration + E2E).
- T-4.1.4 (S) — Rotate-on-leak automation integrated with Vault / Bitwarden / env.

Sub-phase 4.2 — Provenance

- T-4.2.1 (M) — Implement Sigstore signing in Smart-Worker per successful push.
- T-4.2.2 (S) — Store Rekor entry index in `refs/notes/gitpx-attest`.
- T-4.2.3 (S) — `gitpxctl attest verify` command.
- T-4.2.4 (S) — Grafana panel for attestation success rate.

Sub-phase 4.3 — SBOM & Compliance

- T-4.3.1 (M) — Integrate Syft + Grype in receiver / post-push.
- T-4.3.2 (S) — ORT license scan.
- T-4.3.3 (S) — Results in `refs/notes/gitpx-sbom`.
- T-4.3.4 (M) — Custom Gitea middleware surfacing SBOM in UI.

Sub-phase 4.4 — Hardening

- T-4.4.1 (S) — Seccomp + AppArmor profiles.
- T-4.4.2 (S) — Non-root container execution, read-only root FS.
- T-4.4.3 (S) — Distroless final image stage.
- T-4.4.4 (S) — Cosign image signing in release workflow.
- T-4.4.5 (S) — SLSA L3 provenance attached.

Phase 5 — Observability, Notifications & Reports

Sub-phase 5.1 — Metrics & Dashboards

- T-5.1.1 (S) — Expose all metrics from §11.2.
- T-5.1.2 (M) — Build 10 pre-shipped Grafana dashboards.
- T-5.1.3 (S) — Configure alert rules from §11.3.

Sub-phase 5.2 — Notification Service

- T-5.2.1 (L) — Go Notification Service scaffold; subscribe to NATS.
- T-5.2.2 (M) — Template engine + per-user subscription storage.
- T-5.2.3 (M) — Channel adapters: Telegram, Slack, Discord, Matrix, Mattermost, Teams, Email, FCM, APNs, Web Push, ntfy, Gotify, generic webhook.
- T-5.2.4 (M) — Two-way chat bot commands.
- T-5.2.5 (S) — Apprise bridge as fallback for long-tail channels.

Sub-phase 5.3 — Realtime Gateway

- T-5.3.1 (M) — Go Realtime Gateway; WebSocket + SSE endpoints; NATS subscriber.
- T-5.3.2 (S) — Topic filtering + auth.
- T-5.3.3 (S) — Horizontal-scale ready (stateless).

Sub-phase 5.4 — Reports

- T-5.4.1 (M) — Report generator service (Go) using `weasyprint/wkhtmltopdf` backends.
- T-5.4.2 (S) — Daily SLO, weekly security digest, monthly cost reports.
- T-5.4.3 (S) — AI narrative integration (Observability Interpreter agent).

Phase 6 — AI / LLM Layer

Sub-phase 6.1 — Gateway & Local LLM

- T-6.1.1 (M) — Deploy LiteLLM with config from §10.4.
- T-6.1.2 (M) — Deploy Ollama with 2–3 models (qwen2.5-coder:7b, llama3.2:3b, granite-guardian).
- T-6.1.3 (S) — Verify LiteLLM failover chain.
- T-6.1.4 (S) — Budget alerts wired to Alertmanager.

Sub-phase 6.2 — RAG Pipeline

- T-6.2.1 (M) — Qdrant deployment.
- T-6.2.2 (M) — Embedding pipeline (post-push → fastembed → Qdrant).
- T-6.2.3 (S) — Hybrid search endpoint.
- T-6.2.4 (S) — Reranker (bge-v2-m3) integration.

Sub-phase 6.3 — Agents

- T-6.3.1 (L) — PR Reviewer agent.
- T-6.3.2 (M) — Commit Summarizer agent.
- T-6.3.3 (M) — Security Sentinel agent.
- T-6.3.4 (L) — Patch Proposer agent.
- T-6.3.5 (M) — Observability Interpreter.
- T-6.3.6 (L) — Incident Responder (requires action allowlist review).
- T-6.3.7 (M) — Remaining agents (§10.2 rows) — shipped in trickle over 2 sprints.

Sub-phase 6.4 — Safety

- T-6.4.1 (S) — Granite-Guardian input/output classifier integration.
- T-6.4.2 (S) — Prompt audit log in Loki.
- T-6.4.3 (S) — Diff redaction filter.
- T-6.4.4 (S) — Per-repo `ai_external: false` enforcement.

Phase 7 — Testing Pipeline Maturation

(Runs in parallel with 3–6; consolidated here.)

- T-7.1.1 (M) — BATS test suite for Bash SDK.
- T-7.2.1 (L) — k6 scripts (smoke, load, stress, soak).
- T-7.3.1 (M) — Chaos Mesh experiments (kill worker, partition Redis, latency injection).
- T-7.4.1 (M) — OWASP ZAP automation.
- T-7.5.1 (M) — OSS-Fuzz integration.
- T-7.6.1 (M) — Monthly DR drill GitHub Action.
- T-7.7.1 (M) — Contract tests (Pact) against each upstream sandbox.
- T-7.8.1 (M) — Visual regression suite for docs + dashboard.

Phase 8 — GPU / CUDA Acceleration (Optional)

Only when a GPU host is available.

- T-8.1.1 (M) — Provision GPU host (OCI does not offer free GPU; use self-hosted or paid tier).
- T-8.1.2 (M) — NVIDIA Container Toolkit install.
- T-8.2.1 (M) — Switch Ollama to vLLM/TensorRT-LLM.
- T-8.3.1 (M) — CUDA-accelerated embeddings.
- T-8.4.1 (M) — CUDA fingerprint dedup for LFS.
- T-8.5.1 (S) — Benchmarks & dashboards showing speedup.

Phase 9 — Client Applications

Sub-phase 9.1 — Web Dashboard

- T-9.1.1 (L) — Scaffold SvelteKit app; design tokens; base layout.
- T-9.1.2 (L) — Auth integration (OIDC).
- T-9.1.3 (L) — Overview + Repo Detail screens.
- T-9.1.4 (L) — Realtime Push Live View (WebSocket).
- T-9.1.5 (M) — Provenance Explorer.
- T-9.1.6 (M) — Drift Dashboard.
- T-9.1.7 (M) — Security Center.
- T-9.1.8 (M) — AI Reviews.
- T-9.1.9 (M) — Backup & DR.
- T-9.1.10 (M) — Notifications subscription UI.
- T-9.1.11 (L) — YAML editor + Apply (hot-reload) flow.
- T-9.1.12 (S) — Audit Log.
- T-9.1.13 (M) — In-browser terminal.
- T-9.1.14 (M) — i18n (10+ locales).
- T-9.1.15 (M) — WCAG 2.2 AA audit & fixes.
- T-9.1.16 (M) — Playwright E2E suite.

Sub-phase 9.2 — Desktop Apps (Tauri)

- T-9.2.1 (L) — Tauri v2 scaffold, reuse web dashboard code.
- T-9.2.2 (M) — OS notifications + system tray.
- T-9.2.3 (M) — Auto-updater + signed updates.
- T-9.2.4 (S) — Deep integrations ("Open in IDE", URL handler).
- T-9.2.5 (M) — Linux builds (deb/rpm/AppImage/Flatpak/Snap).
- T-9.2.6 (M) — Windows builds (MSIX/NSIS).
- T-9.2.7 (M) — macOS builds (DMG, notarization).
- T-9.2.8 (S) — Homebrew cask.

Sub-phase 9.3 — Android App

- T-9.3.1 (L) — Kotlin + Compose scaffold.
- T-9.3.2 (L) — Core flows (login, status, repo list, push feed).
- T-9.3.3 (M) — Push notifications via FCM.
- T-9.3.4 (M) — Biometric auth.
- T-9.3.5 (M) — Jetpack Glance home-screen widget.
- T-9.3.6 (S) — F-Droid metadata + reproducible build.
- T-9.3.7 (M) — UI tests.

Sub-phase 9.4 — iOS App

- T-9.4.1 (L) — Swift 6 + SwiftUI scaffold.
- T-9.4.2 (L) — Core flows.

- T-9.4.3 (M) — APNs integration.
- T-9.4.4 (M) — Dynamic Island live activity.
- T-9.4.5 (M) — iPad layouts.
- T-9.4.6 (M) — XCUITest suite.

Sub-phase 9.5 — HarmonyOS NEXT App

- T-9.5.1 (L) — ArkTS scaffold.
- T-9.5.2 (L) — Core flows.
- T-9.5.3 (M) — Distributed continuity (phone↔watch↔tablet).
- T-9.5.4 (M) — AppGallery submission.

Sub-phase 9.6 — Aurora OS App

- T-9.6.1 (L) — Qt 6 + QML scaffold.
- T-9.6.2 (L) — Core flows.
- T-9.6.3 (M) — Aurora-specific APIs (notifications, sandboxing).
- T-9.6.4 (M) — Aurora Store submission.

Sub-phase 9.7 — CLI & TUI

- T-9.7.1 (M) — `gitpxctl` per §19.4.
- T-9.7.2 (M) — `gitpxtop` per §19.5.
- T-9.7.3 (S) — Shell completion (bash, zsh, fish, PowerShell).

Sub-phase 9.8 — Browser Extensions

- T-9.8.1 (M) — Chrome/Firefox/Edge MV3 extension.
- T-9.8.2 (S) — Publish on each store.

Sub-phase 9.9 — IDE Extensions

- T-9.9.1 (M) — VS Code extension.
- T-9.9.2 (M) — JetBrains plugin.
- T-9.9.3 (S) — Neovim + Emacs plugins.

Phase 10 — Production Rollout & Launch

- T-10.1 (M) — End-to-end chaos drill.
- T-10.2 (M) — External penetration test.
- T-10.3 (S) — Documentation site (MkDocs Material) at `docs.gitpx.io`.
- T-10.4 (S) — Launch blog post + HN submission + Reddit.
- T-10.5 (S) — Open-source release on Codeberg (irony) + GitHub + our own proxy.
- T-10.6 (M) — Public demo instance (read-only, heavily rate-limited).
- T-10.7 (S) — Status page (Uptime Kuma).
- T-10.8 (M) — 30-day continuous monitoring & triage.

20.x Task Summary Table

Phase	Tasks	Estimated XL-equivalent	Parallelizable Streams
0	11	3 weeks	2
1	21	3 weeks	3
2	14	2 weeks	2
3	37	8 weeks	4
4	18	4 weeks	3
5	14	5 weeks	3
6	17	6 weeks	3
7	8	continuous	ongoing
8	5	2 weeks	1
9	40+	16 weeks	6 (one per client)
10	8	4 weeks	2

Phase	Tasks	Estimated XL-equivalent	Parallelizable Streams
Total	**~193 tasks**	**~25–30 calendar weeks** (with 5-engineer team, 6 parallel streams)	

---

21. Path to Worldwide Scale — From Single VM to Global Mesh

The architecture is designed so that **the same Go binary, the same Redis protocol, and the same config schema** power a hobbyist's Raspberry Pi and a globally distributed production mesh. No code is rewritten as scale increases — only deployment topology changes.

21.1 Three Scale Tiers

Tier	Target Throughput	Infrastructure	Cost
T1 — Single Node	≤ 50 repos, ≤ 500 pushes/day	1× Oracle A1.Flex (4 vCPU / 24 GB)	$0
T2 — Regional HA	≤ 5 000 repos, ≤ 50 000 pushes/day	3× A1.Flex across 2 regions + Backblaze B2 + Cloudflare R2	$0–15/mo
T3 — Global Mesh	500 000+ repos, unlimited	K3s/K8s across 3+ continents, managed Postgres (Neon/Supabase), Dragonfly cluster, NATS JetStream cluster	$200–1 000/mo (still cheap)

21.2 T1 → T2 Migration (Zero Downtime)

Steps, each executable as an Ansible playbook:

1. **Provision second node** in a different Oracle region (e.g., Frankfurt if primary is Amsterdam).
2. **Set up Gitea primary/replica** using built-in HA mode with shared Postgres on Neon free tier.
3. **Redis → Redis Sentinel** (3 nodes: primary, replica, arbiter) or migrate to **Dragonfly** (single binary, handles 1M ops/sec on one node — often removes the need for T2 entirely).
4. **Shared object storage**: migrate Git LFS to Cloudflare R2 (10 GB free, 1 M Class-A ops free) or Backblaze B2.
5. **DNS failover** via Cloudflare Load Balancer (free tier: 2 origins, 1 monitor).
6. **Smart-Workers as stateless pool**: workers claim jobs from the shared Redis stream; adding a new worker is just `systemctl start gitpx-worker`.

21.3 T2 → T3 Migration

1. **Adopt K3s** (lightweight Kubernetes — single binary, runs on the same ARM VMs).
2. **Helm chart** `gitpx/helm/gitpx` (provided in §20 T-7.7) deploys every component with configurable replicas.
3. **Multi-region service mesh**: Cilium (eBPF-based, free) provides L7 policies, mTLS, observability.
4. **Managed data services**:
   - Postgres → Neon (scales to zero) or CockroachDB Serverless (5 GB free).
   - Redis → Dragonfly cluster or Upstash Redis (10 k commands/day free).
   - Object storage → Cloudflare R2 or Backblaze B2.
5. **Global CDN**: Cloudflare in front of both `git://` (via Spectrum, paid) and HTTPS (free).
6. **Federation**: each region runs an independent gitpx cluster. A top-level NATS JetStream mesh replicates events across regions so every cluster sees every push. Users are routed to the nearest cluster via Cloudflare Argo Smart Routing.

21.4 Capacity Planning

Resource	Formula	Example (1 000 repos, 10 000 pushes/day)
Worker vCPU	`(pushes/day × avg_fanout × avg_push_seconds) / 86400 × safety(2)`	`(10 000 × 3 × 8) / 86400 × 2 ≈ 5.5 vCPU`
Worker RAM	`workers × 256 MB + Redis buffer (512 MB) + LFS cache`	`6 × 256 MB + 512 MB + 2 GB ≈ 4 GB`
Storage	`total_repo_size × 1.2 (pack overhead) + LFS objects`	varies
Egress	`pushes/day × avg_push_bytes × fanout × 30`	for 10 000 pushes × 10 MB × 3 × 30 = **9 TB/mo** — use Cloudflare R2 (zero egress fee)

21.5 Egress Cost Engineering

The single biggest surprise cost at scale is **cloud egress**. gitpx mitigates this via:

1. **Cloudflare R2** — zero egress fees, S3-compatible.
2. **Backblaze B2 + Cloudflare Bandwidth Alliance** — free egress via Cloudflare.
3. **Oracle Cloud** — 10 TB/month free egress per tenancy (verify with your region).
4. **Push-once, fan-out via P2P** — experimental: use libp2p to gossip pack files between workers in different regions.
5. **Delta transfer**: only changed objects are pushed; `gitpx` uses `git-upload-pack --thin` wherever supported.
6. **Shallow mirrors**: for upstreams that don't need full history, push with `--depth` and periodic deepen.

21.6 Multi-Tenancy Model (Future)

If gitpx ever becomes a SaaS:

- **Tenant isolation**: namespace per tenant in Gitea + separate Redis prefix + separate K8s namespace.
- **Resource quotas**: CPU/RAM/storage limits enforced via K8s ResourceQuota.
- **Billing**: Stripe + usage metering via Prometheus → OpenMeter (open-source metering).
- **SSO**: Dex/Keycloak + SCIM 2.0 provisioning.
- **Data residency**: tenants can pin to a specific region.

---

22. Change Log & Inconsistency Resolution Record

This section is the **formal audit trail** of every conflict found between the three source documents and the resolution applied in this specification.

22.1 Source Document Versions

ID	File	Version Claim	Date	Size
SRC-A	`Git_Proxy_Idea.md`	v1.0 (base research)	initial	904 lines / 74 KB
SRC-B	`Git_Proxy_Idea_Fixed.md`	"v3.0 — fact-checked"	pass 1	324 lines
SRC-C	`Git_Proxy_Idea_Fixed_2.md`	"v2.1 — independent audit"	pass 2	262 lines

22.2 Inconsistency Resolution Log

#	Topic	SRC-A said	SRC-B said	SRC-C said	SSoT Resolution	Rationale
1	Push command	`git push --mirror`	`git push <refs>`	`git push --force <refs>`	**`git push <explicit-refspecs>` + `--force-with-lease` per-branch-allowed** (§4.4)	`--mirror` deletes refs on upstream unconditionally (data loss). `--force` is destructive. `--force-with-lease` is the safe middle ground.
2	Metric prefix	`git_proxy_*`	`git_proxy_*`	`proxy_*`	**`gitpx_*`** (§11.2)	Clean, short, unique, no collision with other `proxy_*` tools.
3	Version string	implicit v1	"v3.0"	"v2.1"	**v4.0.0 Unified Blueprint**	Supersedes all; uses semver.
4	LLM orchestration	implicit	LiteLLM + LangChain	LiteLLM + direct	**LiteLLM (routing) + LangGraph (orchestration) + DSPy (prompt optimization)** (§10)	Best-of-breed per concern; all active OSS.
5	Event bus	Webhooks only	Redis Streams	Redis Streams + fallback	**Redis Streams primary, NATS JetStream federation, webhooks as escape hatch** (§4.3, §12.2)	Redis for intra-cluster, NATS for inter-region, webhooks for

#	Topic	SRC-A said	SRC-B said	SRC-C said	SSoT Resolution	Rationale
						external integrations.
6	Tunnel / ingress	ngrok hinted	Cloudflare Tunnels	Cloudflare Tunnels	**Cloudflare Tunnels primary, Tailscale Funnel secondary** (§3.2)	Cloudflare: free, zero open ports. Tailscale Funnel: alternative if Cloudflare is blocked (e.g., in some regions).
7	LFS storage	Local only	Dragonfly P2P	Implicit	**Dragonfly P2P for in-cluster + Cloudflare R2 / B2 for origin** (§7.2, §21.5)	P2P eliminates re-download cost, R2 zero egress for public distribution.
8	Signing	Not mentioned	Sigstore/Cosign	Sigstore/Cosign	**Sigstore Cosign for containers + Gitsign for commits + in-toto for attestations** (§7.3, §7.10)	All three layers signed, transparency log via Rekor.
9	Secret scanning	Mentioned	Gitleaks	Gitleaks + TruffleHog	**Gitleaks (fast pre-commit) + TruffleHog (deep, verified) + custom WASM rule plugins** (§7.4)	Two engines catch different classes; WASM plugins allow custom org rules.
10	Risks catalog	R-001..R-008	R-001..R-010	R-001..R-010	**R-001..R-025** (§8)	Added 15 new risks covering supply chain, AI hallucination, GPU contention, regional outages, and more.
11	Language for core	Go (implicit)	Go	Go	**Go (core) + Rust (perf) + CUDA C++ (accel) + TypeScript (web) + platform-native for clients** (§13)	Right tool per layer.
12	Config format	Bash env vars	YAML	YAML	**YAML for declarative, TOML for Gitea `app.ini`, env vars for secrets only** (§4.6, §4.7)	Standard per component.
13	Testing	Unit + integration	Unit/integration/E2E	Unit + chaos	**Full 20-layer matrix** (§18)	Nothing omitted; see §18 matrix.
14	Container runtime	Docker	Docker + Podman	Docker	**Podman default (rootless, daemonless), Docker supported, Firecracker for untrusted code** (§16)	Security-first.
15	Force-push policy	Not specified	Forbidden	Permitted	**Per-branch opt-in via `allow_force: true` + `--force-with-lease`** (§4.4, §4.7)	Flexible, safe, auditable.
16	API auth	None specified	PAT	PAT	**PAT + mTLS + OIDC (SSO) + Sigstore short-lived for CI** (§19.2, §10.6)	Multiple methods per use case.
17	Multi-region	Not covered	Mentioned	Mentioned	**T1/T2/T3 explicit tiers with migration playbooks** (§21)	Concrete path to scale.

#	Topic	SRC-A said	SRC-B said	SRC-C said	SSoT Resolution	Rationale
18	Notification channels	Telegram/Slack/Discord	Same	+ Matrix	**17 channels via Apprise + custom plugins** (§12.3)	Ubiquitous integration.
19	Mobile apps	Android only	Android + iOS	Mentioned	**Android/iOS/HarmonyOS/Aurora + cross-platform KMP fallback** (§19.5–19.9)	Full coverage.
20	Build system	Makefile	Makefile	Taskfile	**Nix flakes top / Bazel polyglot / per-platform native (Gradle, SwiftPM, Cargo)** (§17)	Reproducible + polyglot.

22.3 Net-New Additions (not in any source document)

- **§7.12 WebRTC peer-presence channel** for real-time collaborative repo browsing.
- **§7.13 AI-driven intent diff** — human-readable natural language description of every push.
- **§7.14 OpenCV visual regression testing** for web UIs built on gitpx (client apps).
- **§7.15 CUDA-accelerated pack deduplication** for very large monorepos.
- **§13 full language matrix** including Aurora (Qt/QML) and HarmonyOS NEXT (ArkTS).
- **§14 gitpx-bash-sdk** — reusable bash scripting framework for pipeline authors.
- **§15.3 OpenCV seven-use-case catalog** for visual testing, OCR on screenshots, and more.
- **§17 full bleeding-edge build stack** (Nix/Bazel/Turborepo).
- **§19 full client apps ecosystem** (web, desktop Tauri, 4 mobile platforms, CLI, TUI, browser extensions, 4 editor plugins).
- **§21 three-tier worldwide scale path** with explicit migration playbooks.

---

23. Appendix A — Reference Configurations

A.1 `docker-compose.yml` (T1 single-node starter)

```yaml
version: "3.9"

networks:
  gitpx-internal:
    driver: bridge
  gitpx-edge:
    driver: bridge

volumes:
  gitea-data:
  gitea-config:
  redis-data:
  postgres-data:
  prometheus-data:
  grafana-data:
  loki-data:

services:

  gitea:
    image: gitea/gitea:1.23
    container_name: gitea
    restart: unless-stopped
    environment:
      - USER_UID=1000
      - USER_GID=1000
      - GITEA__database__DB_TYPE=postgres
      - GITEA__database__HOST=postgres:5432
      - GITEA__database__NAME=gitea
```

```yaml
    - GITEA__database__USER=gitea
    - GITEA__database__PASSWD=\${POSTGRES_PASSWORD}
    - GITEA__server__ROOT_URL=https://git.example.com/
    - GITEA__server__SSH_DOMAIN=git.example.com
    - GITEA__webhook__ALLOWED_HOST_LIST=*
  volumes:
    - gitea-data:/data
    - gitea-config:/etc/gitea
    - /etc/timezone:/etc/timezone:ro
    - /etc/localtime:/etc/localtime:ro
  networks: [gitpx-internal, gitpx-edge]
  depends_on:
    - postgres
    - redis

postgres:
  image: postgres:17-alpine
  container_name: postgres
  restart: unless-stopped
  environment:
    - POSTGRES_USER=gitea
    - POSTGRES_PASSWORD=\${POSTGRES_PASSWORD}
    - POSTGRES_DB=gitea
  volumes:
    - postgres-data:/var/lib/postgresql/data
  networks: [gitpx-internal]
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U gitea"]
    interval: 10s
    retries: 5

redis:
  # For prod, prefer Dragonfly: docker.dragonflydb.io/dragonflydb/dragonfly
  image: redis:7-alpine
  container_name: redis
  restart: unless-stopped
  command: redis-server --save 60 1 --loglevel warning --appendonly yes
  volumes:
    - redis-data:/data
  networks: [gitpx-internal]
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s

gitpx-worker:
  image: ghcr.io/vasic-digital/gitpx-worker:latest
  container_name: gitpx-worker
  restart: unless-stopped
  environment:
    - GITPX_REDIS_URL=redis://redis:6379
    - GITPX_GITEA_URL=http://gitea:3000
    - GITPX_GITEA_TOKEN=\${GITPX_GITEA_TOKEN}
    - GITPX_CONFIG=/etc/gitpx/upstreams.yaml
    - GITPX_LOG_LEVEL=info
    - GITPX_METRICS_ADDR=:9100
```

```yaml
    - PASSWORD_GITHUB=\${PASSWORD_GITHUB}
    - PASSWORD_GITLAB=\${PASSWORD_GITLAB}
    - PASSWORD_CODEBERG=\${PASSWORD_CODEBERG}
  volumes:
    - ./config/upstreams.yaml:/etc/gitpx/upstreams.yaml:ro
  networks: [gitpx-internal]
  depends_on: [redis, gitea]
  deploy:
    replicas: 2   # scale horizontally; each worker claims jobs from the stream

caddy:
  image: caddy:2-alpine
  container_name: caddy
  restart: unless-stopped
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./config/Caddyfile:/etc/caddy/Caddyfile:ro
    - caddy-data:/data
    - caddy-config:/config
  networks: [gitpx-edge]
  depends_on: [gitea]

cloudflared:
  image: cloudflare/cloudflared:latest
  container_name: cloudflared
  restart: unless-stopped
  command: tunnel --no-autoupdate run
  environment:
    - TUNNEL_TOKEN=\${CF_TUNNEL_TOKEN}
  networks: [gitpx-edge]

prometheus:
  image: prom/prometheus:latest
  container_name: prometheus
  restart: unless-stopped
  volumes:
    - ./config/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - ./config/alerts:/etc/prometheus/alerts:ro
    - prometheus-data:/prometheus
  networks: [gitpx-internal]

grafana:
  image: grafana/grafana:latest
  container_name: grafana
  restart: unless-stopped
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=\${GRAFANA_PASSWORD}
  volumes:
    - grafana-data:/var/lib/grafana
    - ./config/grafana-dashboards:/var/lib/grafana/dashboards:ro
  networks: [gitpx-internal]

loki:
```

```yaml
    image: grafana/loki:latest
    container_name: loki
    restart: unless-stopped
    command: -config.file=/etc/loki/config.yaml
    volumes:
      - ./config/loki.yaml:/etc/loki/config.yaml:ro
      - loki-data:/loki
    networks: [gitpx-internal]

volumes:
  caddy-data:
  caddy-config:
```

A.2 `Caddyfile`

```
{
    email admin@example.com
    servers {
        metrics
    }
}

git.example.com {
    encode zstd gzip
    reverse_proxy gitea:3000
    header {
        Strict-Transport-Security "max-age=31536000; includeSubDomains; preload"
        X-Content-Type-Options "nosniff"
        X-Frame-Options "DENY"
        Referrer-Policy "strict-origin-when-cross-origin"
        Permissions-Policy "interest-cohort=()"
    }
    log {
        output file /var/log/caddy/access.log
        format json
    }
}

metrics.example.com {
    encode zstd gzip
    basicauth {
        admin {env.METRICS_BASIC_AUTH_HASH}
    }
    reverse_proxy grafana:3000
}
```

A.3 `cloudflared` tunnel config (`~/.cloudflared/config.yml`)

```yaml
tunnel: <TUNNEL-UUID>
credentials-file: /etc/cloudflared/<TUNNEL-UUID>.json

ingress:
  - hostname: git.example.com
```

```
    service: http://caddy:80
    originRequest:
      noTLSVerify: false
      http2Origin: true
  - hostname: ssh.git.example.com
    service: ssh://gitea:22
  - service: http_status:404
```

A.4 prometheus.yml

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    cluster: gitpx-primary
    replica: '0'

rule_files:
  - /etc/prometheus/alerts/*.yml

alerting:
  alertmanagers:
    - static_configs:
        - targets: ['alertmanager:9093']

scrape_configs:
  - job_name: gitpx-worker
    static_configs:
      - targets: ['gitpx-worker:9100']
    metric_relabel_configs:
      - source_labels: [__name__]
        regex: 'gitpx_.*'
        action: keep

  - job_name: gitea
    static_configs:
      - targets: ['gitea:3000']
    metrics_path: /metrics
    authorization:
      credentials_file: /etc/prometheus/gitea-metrics-token

  - job_name: node-exporter
    static_configs:
      - targets: ['node-exporter:9100']

  - job_name: redis
    static_configs:
      - targets: ['redis-exporter:9121']

  - job_name: postgres
    static_configs:
      - targets: ['postgres-exporter:9187']

  - job_name: cadvisor
```

```
    static_configs:
      - targets: ['cadvisor:8080']
```

A.5 Nginx fallback config (if Caddy unsuitable)

```
server {
    listen 443 ssl http2;
    server_name git.example.com;

    ssl_certificate     /etc/letsencrypt/live/git.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/git.example.com/privkey.pem;
    ssl_protocols       TLSv1.3 TLSv1.2;
    ssl_ciphers         HIGH:!aNULL:!MD5;

    client_max_body_size 1G;     # allow large git pushes / LFS
    proxy_read_timeout   600s;
    proxy_send_timeout   600s;

    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header X-Content-Type-Options "nosniff" always;

    location / {
        proxy_pass         http://127.0.0.1:3000;
        proxy_set_header   Host              \$host;
        proxy_set_header   X-Real-IP         \$remote_addr;
        proxy_set_header   X-Forwarded-For   \$proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto \$scheme;
        proxy_buffering    off;  # critical for git-upload-pack streaming
    }
}
```

A.6 Sample `upstreams.yaml`

```yaml
apiVersion: gitpx.io/v1
kind: UpstreamSet
metadata:
  name: default
  scope: global
spec:
  defaults:
    timeout: 120s
    retries: 5
    backoff:
      initial: 5s
      max: 300s
      multiplier: 2
    force_with_lease:
      allowed_branches: []        # explicit opt-in
    mirror_tags: true
    mirror_lfs: true
    sign_commits: false           # opt-in via Gitsign
  upstreams:
    github:
```

```yaml
      enabled: true
      type: github
      url_template: "https://github.com/\${org}/\${repo}.git"
      auth:
        method: token
        token_env: PASSWORD_GITHUB
      metadata_sync:
        issues: false             # issues owned by gitpx-primary
        releases: true
        topics: true
      rate_limit:
        rps: 5
        burst: 10
    gitlab:
      enabled: true
      type: gitlab
      url_template: "https://gitlab.com/\${org}/\${repo}.git"
      auth:
        method: token
        token_env: PASSWORD_GITLAB
    codeberg:
      enabled: true
      type: gitea
      base_url: "https://codeberg.org"
      url_template: "\${base_url}/\${org}/\${repo}.git"
      auth:
        method: token
        token_env: PASSWORD_CODEBERG
    sourcehut:
      enabled: false
      type: sourcehut
      url_template: "https://git.sr.ht/~\${org}/\${repo}"
      auth:
        method: ssh_key
        ssh_key_file: /var/lib/gitpx/keys/sourcehut.ed25519
```

A.7 Per-repo `.gitpx/upstreams.yaml` (override)

```yaml
apiVersion: gitpx.io/v1
kind: UpstreamSet
metadata:
  name: myrepo-override
  scope: repo
spec:
  upstreams:
    github:
      enabled: true
      org: myorg              # override org for this repo only
      repo: myrepo
    sourcehut:
      enabled: true            # turn on sourcehut for this repo
      org: me
      repo: myrepo
  branch_policies:
```

```
    main:
      allow_force: false
      require_signature: true
    release/*:
      allow_force: false
      require_signature: true
    feature/*:
      allow_force: true        # per-branch force-push allowed
      force_strategy: with_lease
```

A.8 Alertmanager routing

```
route:
  receiver: default
  group_by: [alertname, severity, upstream]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  routes:
    - matchers: [severity="critical"]
      receiver: pager
      continue: true
    - matchers: [severity="warning"]
      receiver: chat
    - matchers: [severity="info"]
      receiver: log_only

receivers:
  - name: default
    webhook_configs:
      - url: http://gitpx-notifier:9200/alerts
  - name: pager
    webhook_configs:
      - url: http://gitpx-notifier:9200/alerts/pager
  - name: chat
    webhook_configs:
      - url: http://gitpx-notifier:9200/alerts/chat
  - name: log_only
    webhook_configs:
      - url: http://gitpx-notifier:9200/alerts/log
```

---

24. Appendix B — Reference Code Snippets

> **Purpose**: boilerplate that engineers can copy on day one. Not exhaustive; this specification sits beside the code repo.

B.1 Go smart-worker skeleton (`cmd/gitpx-worker/main.go`)

```go
// Copyright 2026 vasic-digital.
// SPDX-License-Identifier: Apache-2.0

package main

import (
```

```go
    "context"
    "flag"
    "log/slog"
    "os"
    "os/signal"
    "syscall"

    "github.com/vasic-digital/gitpx/internal/config"
    "github.com/vasic-digital/gitpx/internal/observability"
    "github.com/vasic-digital/gitpx/internal/runtime"
)

func main() {
    cfgPath := flag.String("config", "/etc/gitpx/upstreams.yaml", "path to upstream config")
    logLevel := flag.String("log-level", "info", "log level")
    flag.Parse()

    logger := observability.NewLogger(*logLevel)
    slog.SetDefault(logger)

    cfg, err := config.Load(*cfgPath)
    if err != nil {
        logger.Error("failed to load config", "err", err)
        os.Exit(1)
    }

    ctx, cancel := signal.NotifyContext(context.Background(),
        syscall.SIGINT, syscall.SIGTERM)
    defer cancel()

    metrics := observability.MustRegisterMetrics()
    tracer := observability.MustInitTracer(ctx, "gitpx-worker")
    defer tracer.Shutdown(context.Background())

    rt, err := runtime.New(cfg, metrics, logger)
    if err != nil {
        logger.Error("failed to build runtime", "err", err)
        os.Exit(1)
    }

    if err := rt.Run(ctx); err != nil {
        logger.Error("runtime exited with error", "err", err)
        os.Exit(1)
    }
    logger.Info("shutdown complete")
}
```

B.2 Safe-push implementation (`internal/push/push.go`)

```go
package push

import (
    "context"
    "errors"
```

```go
    "fmt"
    "os/exec"
    "time"

    "github.com/vasic-digital/gitpx/internal/gitutil"
)

type Policy struct {
    Upstream          string
    Branch            string
    ExpectedUpstream  string    // SHA we believe upstream currently has
    AllowForce        bool
    ForceStrategy     string    // "with_lease" | "never"
    Timeout           time.Duration
}

type Pusher struct {
    RunCmd func(context.Context, string, ...string) ([]byte, []byte, error)
}

var ErrUnsafePush = errors.New("refusing unsafe push")

func (p Pusher) Push(ctx context.Context, workDir, authURL string, pol Policy) error {
    ctx, cancel := context.WithTimeout(ctx, pol.Timeout)
    defer cancel()

    args := []string{"-C", workDir, "push"}
    if pol.AllowForce {
        if pol.ForceStrategy != "with_lease" {
            return fmt.Errorf("%w: allow_force without with_lease", ErrUnsafePush)
        }
        if pol.ExpectedUpstream == "" {
            return fmt.Errorf("%w: with_lease requires expected upstream SHA", ErrUnsafePush)
        }
        args = append(args,
            fmt.Sprintf("--force-with-lease=refs/heads/%s:%s",
                pol.Branch, pol.ExpectedUpstream))
    }
    args = append(args, authURL,
        fmt.Sprintf("refs/heads/%s:refs/heads/%s", pol.Branch, pol.Branch))

    stdout, stderr, err := p.RunCmd(ctx, "git", args...)
    if err != nil {
        return fmt.Errorf("git push failed: %w: stdout=%q stderr=%q",
            err, stdout, stderr)
    }
    return nil
}

// helper: run under default exec
func DefaultRun(ctx context.Context, name string, args ...string) ([]byte, []byte, error) {
    cmd := exec.CommandContext(ctx, name, args...)
    var stdout, stderr []byte
    cmd.Stdout = (*bufwriter)(&stdout)
    cmd.Stderr = (*bufwriter)(&stderr)
```

```go
    err := cmd.Run()
    return stdout, stderr, err
}

type bufwriter []byte

func (b *bufwriter) Write(p []byte) (int, error) {
    *b = append(*b, p...)
    return len(p), nil
}
```

B.3 Webhook receiver with HMAC verification

```go
package webhook

import (
    "crypto/hmac"
    "crypto/sha256"
    "encoding/hex"
    "errors"
    "io"
    "net/http"
)

var ErrBadSignature = errors.New("invalid webhook signature")

func VerifyGitea(req *http.Request, secret []byte) ([]byte, error) {
    sig := req.Header.Get("X-Gitea-Signature")
    if sig == "" {
        return nil, ErrBadSignature
    }
    body, err := io.ReadAll(req.Body)
    if err != nil {
        return nil, err
    }
    mac := hmac.New(sha256.New, secret)
    mac.Write(body)
    expected := hex.EncodeToString(mac.Sum(nil))
    if !hmac.Equal([]byte(sig), []byte(expected)) {
        return nil, ErrBadSignature
    }
    return body, nil
}
```

B.4 Sigstore Cosign keyless sign (CI)

```bash
#!/usr/bin/env bash
set -euo pipefail

IMAGE="\${1:?image ref required}"

# Assumes OIDC token is present (GitHub Actions, GitLab CI, etc.)
COSIGN_EXPERIMENTAL=1 cosign sign --yes \
```

```
    --rekor-url "https://rekor.sigstore.dev" \
    --oidc-issuer "https://token.actions.githubusercontent.com" \
    "\$IMAGE"

cosign verify \
    --certificate-identity-regexp "https://github.com/vasic-digital/.*" \
    --certificate-oidc-issuer "https://token.actions.githubusercontent.com" \
    "\$IMAGE"
```

B.5 Redis Stream consumer (Go)

```go
func (w *Worker) consume(ctx context.Context) error {
    streamKey := fmt.Sprintf("gitpx:events:%d:%s", w.repoID, w.upstream)
    group := fmt.Sprintf("gitpx:workers:%s", w.upstream)
    consumer := w.id

    _ = w.rdb.XGroupCreateMkStream(ctx, streamKey, group, "0").Err()

    for {
        select {
        case <-ctx.Done():
            return ctx.Err()
        default:
        }
        res, err := w.rdb.XReadGroup(ctx, &redis.XReadGroupArgs{
            Group:    group,
            Consumer: consumer,
            Streams:  []string{streamKey, ">"},
            Count:    8,
            Block:    5 * time.Second,
        }).Result()
        if err != nil {
            if errors.Is(err, redis.Nil) {
                continue
            }
            return err
        }
        for _, s := range res {
            for _, m := range s.Messages {
                if err := w.handle(ctx, m); err != nil {
                    w.logger.Error("handle failed", "id", m.ID, "err", err)
                    // don't ack: message will be retried or DLQ'd via PEL
                    continue
                }
                w.rdb.XAck(ctx, streamKey, group, m.ID)
            }
        }
    }
}
```

B.6 Firecracker microVM runner for untrusted AI patches

```go
func (r *Runner) RunInFirecracker(ctx context.Context, patch []byte) (*Report, error) {
    workDir, err := os.MkdirTemp("", "gitpx-fc-*")
    if err != nil { return nil, err }
    defer os.RemoveAll(workDir)

    if err := os.WriteFile(filepath.Join(workDir, "patch.diff"), patch, 0600); err != nil {
        return nil, err
    }

    cfg := firecracker.Config{
        SocketPath:      filepath.Join(workDir, "fc.sock"),
        KernelImagePath: "/var/lib/gitpx/fc/vmlinux",
        KernelArgs:      "console=ttyS0 reboot=k panic=1 pci=off",
        Drives: []models.Drive{{
            DriveID:      firecracker.String("rootfs"),
            IsRootDevice: firecracker.Bool(true),
            IsReadOnly:   firecracker.Bool(false),
            PathOnHost:   firecracker.String("/var/lib/gitpx/fc/rootfs.ext4"),
        }},
        MachineCfg: models.MachineConfiguration{
            VcpuCount:  firecracker.Int64(1),
            MemSizeMib: firecracker.Int64(512),
        },
        NetworkInterfaces: firecracker.NetworkInterfaces{
            firecracker.NetworkInterface{
                StaticConfiguration: &firecracker.StaticNetworkConfiguration{
                    MacAddress:  "AA:FC:00:00:00:01",
                    HostDevName: "tap-gitpx",
                },
            },
        },
    }
    m, err := firecracker.NewMachine(ctx, cfg)
    if err != nil { return nil, err }
    if err := m.Start(ctx); err != nil { return nil, err }
    defer m.StopVMM()

    return r.collectReport(ctx, workDir)
}
```

---

25. Appendix C — Open Source Tool Inventory

This is the complete catalog of every third-party tool referenced in this specification. Each entry is pinned at a stable release as of writing; versions should be re-verified at integration time (see §0 conventions).

C.1 Core Runtime

Tool	Version	License	URL	Role
Gitea	≥ 1.23	MIT	https://gitea.com	Presentation layer, UI, API
Forgejo	≥ 10 (alt.)	MIT/GPL	https://forgejo.org	Drop-in alt. to Gitea
Go	1.23+	BSD	https://go.dev	Smart-worker language
PostgreSQL	16+	PostgreSQL	https://postgresql.org	Metadata

Tool	Version	License	URL	Role
Redis	7+ / Dragonfly	BSD / BSL	https://redis.io / https://dragonflydb.io	Event bus, cache
NATS JetStream	2.10+	Apache-2.0	https://nats.io	Inter-region events

C.2 Ingress / Network

Tool	License	URL	Role
Cloudflare Tunnel (cloudflared)	Apache-2.0	https://github.com/cloudflare/cloudflared	Zero-port ingress
Tailscale Funnel	BSD/proprietary	https://tailscale.com	Alternative ingress
Caddy	Apache-2.0	https://caddyserver.com	Reverse proxy
Traefik	MIT	https://traefik.io	Alternative reverse proxy
Nginx	BSD	https://nginx.org	Fallback reverse proxy

C.3 Security / Supply Chain

Tool	License	URL	Role
Sigstore Cosign	Apache-2.0	https://sigstore.dev	Container signing
Sigstore Gitsign	Apache-2.0	https://github.com/sigstore/gitsign	Commit signing
Rekor	Apache-2.0	https://github.com/sigstore/rekor	Transparency log
in-toto	Apache-2.0	https://in-toto.io	Attestations
SLSA	CC-BY-4.0	https://slsa.dev	Build provenance
Gitleaks	MIT	https://github.com/gitleaks/gitleaks	Secret scan (fast)
TruffleHog	AGPL	https://github.com/trufflesecurity/trufflehog	Secret scan (verified)
Trivy	Apache-2.0	https://trivy.dev	Container + IaC scan
Grype	Apache-2.0	https://github.com/anchore/grype	CVE scan
Syft	Apache-2.0	https://github.com/anchore/syft	SBOM
CycloneDX	Apache-2.0	https://cyclonedx.org	SBOM format
OpenSCAP	LGPL	https://www.open-scap.org	Compliance scan
Falco	Apache-2.0	https://falco.org	Runtime security
Teleport	Apache-2.0	https://goteleport.com	Session audit
Wazuh	GPLv2	https://wazuh.com	SIEM

C.4 Observability

Tool	License	URL	Role
Prometheus	Apache-2.0	https://prometheus.io	Metrics
Grafana	AGPLv3	https://grafana.com	Dashboards
Loki	AGPLv3	https://grafana.com/oss/loki	Logs
Tempo	AGPLv3	https://grafana.com/oss/tempo	Traces
Pyroscope	Apache-2.0	https://pyroscope.io	Profiling
Alertmanager	Apache-2.0	https://prometheus.io	Alert routing
OpenTelemetry	Apache-2.0	https://opentelemetry.io	Unified telemetry
Vector	MPL-2.0	https://vector.dev	Log pipeline

Tool	License	URL	Role
eBPF / Cilium	Apache-2.0	https://cilium.io	Kernel telemetry
Pixie	Apache-2.0	https://px.dev	eBPF observability
Parca	Apache-2.0	https://parca.dev	Continuous profiling
Uptime Kuma	MIT	https://uptime.kuma.pet	Status page

C.5 AI / LLM

Tool	License	URL	Role
LiteLLM	MIT	https://github.com/BerriAI/litellm	Multi-provider router
LangGraph	MIT	https://github.com/langchain-ai/langgraph	Multi-agent orchestration
DSPy	MIT	https://github.com/stanfordnlp/dspy	Prompt compilation
Ollama	MIT	https://ollama.com	Local LLM runtime
llama.cpp	MIT	https://github.com/ggerganov/llama.cpp	Quantized inference
vLLM	Apache-2.0	https://github.com/vllm-project/vllm	High-throughput inference
TensorRT-LLM	Apache-2.0	https://github.com/NVIDIA/TensorRT-LLM	GPU inference
Qdrant	Apache-2.0	https://qdrant.tech	Vector DB
ChromaDB	Apache-2.0	https://trychroma.com	Vector DB (alt.)
pgvector	PostgreSQL	https://github.com/pgvector/pgvector	Postgres vector ext.
Guardrails	Apache-2.0	https://guardrailsai.com	LLM output guards
NeMo Guardrails	Apache-2.0	https://github.com/NVIDIA/NeMo-Guardrails	LLM safety

C.6 Build / Packaging

Tool	License	URL	Role
Nix / NixOS	LGPL	https://nixos.org	Reproducible builds
Bazel	Apache-2.0	https://bazel.build	Polyglot builds
Turborepo	MPL-2.0	https://turbo.build	JS monorepo
pnpm	MIT	https://pnpm.io	JS package manager
Gradle	Apache-2.0	https://gradle.org	JVM/Android build
SwiftPM	Apache-2.0	https://swift.org	Apple build
Cargo	MIT/Apache	https://doc.rust-lang.org/cargo	Rust build
Goreleaser	MIT	https://goreleaser.com	Go release automation
ko	Apache-2.0	https://ko.build	Go → container
Melange + apko	Apache-2.0	https://edu.chainguard.dev	Distroless builds
Task	MIT	https://taskfile.dev	Task runner
just	CC0	https://github.com/casey/just	Task runner (alt.)
Renovate	AGPLv3	https://github.com/renovatebot/renovate	Dep updates

C.7 Testing

Tool	License	URL	Role
gotestsum	MIT	https://github.com/gotestyourself/gotestsum	Go test UX

Tool	License	URL	Role
testcontainers-go	MIT	https://testcontainers.com	Integration harness
Pact	MIT	https://pact.io	Contract tests
Playwright	Apache-2.0	https://playwright.dev	Browser E2E
Cypress	MIT	https://cypress.io	Browser E2E (alt.)
k6	AGPLv3	https://k6.io	Load tests
Litmus	Apache-2.0	https://litmuschaos.io	Chaos
Chaos Mesh	Apache-2.0	https://chaos-mesh.org	Chaos (alt.)
Gremlin	proprietary	https://gremlin.com	Chaos (commercial)
go-fuzz / native fuzz	BSD	https://go.dev	Go fuzzing
BATS	MIT	https://github.com/bats-core/bats-core	Bash tests
shellcheck	GPLv3	https://shellcheck.net	Bash linter
shfmt	BSD	https://github.com/mvdan/sh	Bash formatter
kcov	GPL	https://github.com/SimonKagstrom/kcov	Bash coverage
ZAP	Apache-2.0	https://zaproxy.org	DAST
Nuclei	MIT	https://github.com/projectdiscovery/nuclei	Vulnerability scanner
Semgrep	LGPL	https://semgrep.dev	SAST

C.8 Container / Virtualization

Tool	License	URL	Role
Podman	Apache-2.0	https://podman.io	Rootless containers
Docker Engine	Apache-2.0	https://docker.com	Containers (alt.)
containerd	Apache-2.0	https://containerd.io	Runtime
CRI-O	Apache-2.0	https://cri-o.io	K8s runtime
Buildx / BuildKit	Apache-2.0	https://github.com/moby/buildkit	Builds
Firecracker	Apache-2.0	https://firecracker-microvm.github.io	microVMs
Kata Containers	Apache-2.0	https://katacontainers.io	VM-isolated containers
gVisor	Apache-2.0	https://gvisor.dev	User-space kernel
QEMU / KVM	GPLv2	https://qemu.org	Virtualization
libvirt	LGPL	https://libvirt.org	VM management
K3s	Apache-2.0	https://k3s.io	Lightweight K8s
K0s	Apache-2.0	https://k0sproject.io	Lightweight K8s (alt.)

C.9 Front-End / Clients

Tool	License	URL	Role
SvelteKit	MIT	https://svelte.dev	Web framework
React	MIT	https://react.dev	Web framework (alt.)
TailwindCSS	MIT	https://tailwindcss.com	Styles
shadcn/ui	MIT	https://ui.shadcn.com	Components
Tauri v2	MIT	https://tauri.app	Desktop

Tool	License	URL	Role
Jetpack Compose	Apache-2.0	https://developer.android.com	Android UI
SwiftUI	Apache-2.0	https://developer.apple.com	iOS UI
ArkTS / ArkUI	Apache-2.0	https://developer.harmonyos.com	HarmonyOS NEXT
Qt / QML	LGPL / commercial	https://qt.io	Aurora OS
Kotlin Multiplatform	Apache-2.0	https://kotlinlang.org/lp/mobile/	Shared mobile logic
Flutter	BSD	https://flutter.dev	Cross-platform (alt.)

C.10 CUDA / GPU

Tool	License	URL	Role
CUDA Toolkit	NVIDIA EULA	https://developer.nvidia.com/cuda	GPU SDK
cuOpt	NVIDIA EULA	https://developer.nvidia.com/cuopt	Optimization
cuDNN	NVIDIA EULA	https://developer.nvidia.com/cudnn	DNN accel.
OpenCV CUDA	Apache-2.0	https://opencv.org	Vision accel.
NCCL	BSD	https://developer.nvidia.com/nccl	Multi-GPU
Triton Inference	Apache-2.0	https://github.com/triton-inference-server/server	Model serving

---

26. Appendix D — Cloud Free Tier Registry

> **Caveat**: every provider revises free tiers without notice. Verify the current terms before committing. This table reflects publicly-posted terms at the time of writing; treat as **[VERIFY-AT-INTEGRATION]**.

Provider	Offering	Free Quota	Notes
Oracle Cloud	A1.Flex (Ampere Arm)	4 OCPU / 24 GB RAM / 200 GB block	“Always free”; region-dependent capacity
Oracle Cloud	Egress	10 TB/mo	Per-tenancy
Cloudflare	Tunnel / Workers / Pages	generous	CDN, serverless, pages
Cloudflare	R2 object storage	10 GB + 1 M Class-A + 10 M Class-B ops/mo + zero egress	S3-compatible
Backblaze	B2	10 GB free + free egress via Cloudflare	S3-compatible
Neon	Postgres	0.5 GB storage + scale-to-zero compute	
Supabase	Postgres + Auth + Storage	500 MB DB + 1 GB storage	
CockroachDB Serverless	Postgres-compatible	5 GB storage + 50 M request units/mo	
Upstash	Redis	10 k commands/day	Global replication
Grafana Cloud	Metrics/Logs/Traces	10 k metrics, 50 GB logs, 50 GB traces	Free forever
Sentry	Error tracking	5 k errors/mo	
Better Stack / Logtail	Logs	limited	
Fly.io	VMs	3 shared-cpu-1x / 256 MB	
Render	Web services	750 h/mo	
Railway	VMs	$5 free credit	Limited
GitHub Actions	CI minutes	2 000 min/mo (public: unlimited)	

Provider	Offering	Free Quota	Notes
Gitea Actions	Self-hosted	Unlimited	Run on Oracle A1.Flex
Drone CI	Self-hosted	Unlimited	
Let's Encrypt	TLS certs	Unlimited	
ZeroSSL	TLS certs	90 free certs/yr	
NTFY	Push notifications	self-hosted free	
MinIO	Self-hosted object storage	Unlimited	
Tailscale	VPN	3 users / 100 devices	
Vercel / Netlify	Static + serverless	Generous hobby plans	Front-end hosting

---

27. Appendix E — Full Risk Register (R-001 … R-025)

ID	Risk	Likelihood	Impact	Owner	Review cadence	Status	Mitigation →
R-001	Network partition between proxy and upstream	High	Low	SRE	monthly	Mitigated	§9.1
R-002	Upstream API rate-limit exhaustion	High	Medium	Platform	weekly	Mitigated	§9.2
R-003	Force push data loss	Medium	High	Git-core	monthly	Mitigated	§9.3
R-004	Credential leakage	Medium	Critical	Security	monthly	Mitigated	§9.4
R-005	Push reordering / race	Medium	Medium	Git-core	monthly	Mitigated	§9.5
R-006	Disk exhaustion via large pushes	Medium	High	SRE	weekly	Mitigated	§9.6
R-007	Single-node failure	Medium	High	SRE	monthly	Mitigated	§9.7 / §21
R-008	Operator error	High	Medium	Leads	monthly	Mitigated	§9.8
R-009	LFS unsupported on upstream	Medium	Medium	Platform	release	Mitigated	§9.9
R-010	Webhook replay / spoof	Medium	High	Security	monthly	Mitigated	§9.10
R-011	Supply-chain attack on dependencies	Low	Critical	Security	weekly	Mitigated	§9.11
R-012	AI patch introduces vulnerability	Medium	High	AI team	per-incident	Mitigated	§9.12
R-013	AI hallucinated commit message	High	Low	AI team	monthly	Mitigated	§9.13
R-014	GPU contention (noisy neighbor)	Medium	Medium	SRE	monthly	Mitigated	§9.14
R-015	Regional cloud outage	Low	High	SRE	quarterly	Mitigated	§9.15 / §21
R-016	Key compromise (signing, SSH)	Low	Critical	Security	monthly	Mitigated	§9.16

ID	Risk	Likelihood	Impact	Owner	Review cadence	Status	Mitigation →
R-017	Denial of service on ingress	Medium	High	Security	monthly	Mitigated	§9.17
R-018	Misconfigured upstream policy	High	Medium	Platform	monthly	Mitigated	§9.18
R-019	Runaway log/metric cardinality	Medium	Medium	SRE	monthly	Mitigated	§9.19
R-020	Legal: DMCA / export controls	Low	High	Legal	per-incident	Acknowledged	§9.20
R-021	Repo corruption / pack poisoning	Low	Critical	Git-core	monthly	Mitigated	§9.21
R-022	Time drift breaking signatures	Low	Medium	SRE	quarterly	Mitigated	§9.22
R-023	TLS cert expiration	Low	High	SRE	monthly	Mitigated	§9.23
R-024	Container escape	Low	Critical	Security	quarterly	Mitigated	§9.24
R-025	Governance / single maintainer bus factor	Medium	High	Leadership	quarterly	Partial	§9.25

---

28. Appendix F — Metrics Catalog with Queries

Full list of `gitpx_*` metrics with example PromQL queries.

Metric	Type	Labels	Meaning
`gitpx_push_total`	counter	`upstream`, `repo`, `status`	Pushes attempted
`gitpx_push_duration_seconds`	histogram	`upstream`, `repo`	End-to-end push latency
`gitpx_push_bytes_total`	counter	`upstream`, `repo`, `direction`	Wire bytes in/out
`gitpx_queue_depth`	gauge	`upstream`	Pending jobs per upstream
`gitpx_worker_busy`	gauge	`worker_id`	0/1 per worker
`gitpx_worker_heartbeat_timestamp_seconds`	gauge	`worker_id`	Unix ts of last heartbeat
`gitpx_retry_total`	counter	`upstream`, `reason`	Retry reasons breakdown
`gitpx_deadletter_total`	counter	`upstream`, `reason`	Jobs moved to DLQ
`gitpx_auth_failure_total`	counter	`upstream`, `auth_method`	Auth failures
`gitpx_lfs_bytes_total`	counter	`direction`, `backend`	LFS transfer
`gitpx_lfs_cache_hits_total`	counter	`backend`	Local cache effectiveness
`gitpx_rate_limit_remaining`	gauge	`upstream`	Remaining tokens
`gitpx_ai_calls_total`	counter	`provider`, `model`, `role`	LLM call breakdown
`gitpx_ai_tokens_total`	counter	`provider`, `model`, `kind`	`kind={prompt,completion}`
`gitpx_ai_cost_usd_total`	counter	`provider`, `model`	Running cost
`gitpx_ai_latency_seconds`	histogram	`provider`, `model`	First-token + total
`gitpx_event_dispatch_total`	counter	`channel`, `status`	Notifications
`gitpx_webhook_signature_failure_total`	counter	`source`	Bad HMAC
`gitpx_force_push_total`	counter	`upstream`, `repo`, `branch`	Force-pushes performed
`gitpx_force_push_blocked_total`	counter	`upstream`, `repo`, `reason`	Safety blocks

Metric	Type	Labels	Meaning
`gitpx_pack_dedupe_bytes_saved_total`	counter	`repo`	CUDA dedupe savings
`gitpx_scan_findings_total`	counter	`scanner`, `severity`	Security scan results
`gitpx_signature_verified_total`	counter	`kind`, `upstream`	Cosign/Gitsign verifications
`gitpx_tunnel_up`	gauge	`tunnel_name`	0/1 Cloudflare tunnel health
`gitpx_redis_stream_pel`	gauge	`stream`, `group`	Pending entries list size
`gitpx_build_info`	gauge	`version`, `commit`, `go_version`	Constant 1
`gitpx_config_reload_total`	counter	`status`	Config reloads

F.1 Useful queries

```
# 95p push latency per upstream
histogram_quantile(0.95,
  sum by (le, upstream) (rate(gitpx_push_duration_seconds_bucket[5m])))

# Error rate per upstream
sum by (upstream) (rate(gitpx_push_total{status="error"}[5m]))
  / sum by (upstream) (rate(gitpx_push_total[5m]))

# LLM spend per provider per day
sum by (provider) (increase(gitpx_ai_cost_usd_total[1d]))

# Force-push safety: ratio of blocked to attempted
sum(rate(gitpx_force_push_blocked_total[1h]))
  / sum(rate(gitpx_force_push_total[1h]) + rate(gitpx_force_push_blocked_total[1h]))

# Worker health: any worker whose heartbeat lags
time() - max by (worker_id) (gitpx_worker_heartbeat_timestamp_seconds) > 60
```

---

29. Appendix G — Alert Runbook Library

Each alert defined in §11.3 has a matching runbook here. Each runbook is structured as: **Trigger**, **Impact**, **Diagnose**, **Mitigate**, **Follow-up**.

G.1 `GitpxPushFailureRateHigh`

- **Trigger**: `sum(rate(gitpx_push_total{status="error"}[5m])) / sum(rate(gitpx_push_total[5m])) > 0.05` for 10 min.
- **Impact**: Mirror drift; downstream consumers get stale refs.
- **Diagnose**:
  1. Run `gitpxctl status --upstream=<u>` — check error breakdown.
  2. `kubectl logs -n gitpx -l app=gitpx-worker --tail=200`.
  3. Inspect Grafana panel “Push errors by reason”.
- **Mitigate**:
  - If `auth` errors dominate → rotate PAT via `gitpxctl rotate-token <upstream>`.
  - If `rate_limit` → temporarily increase backoff via config hot-reload.
  - If `network` → verify Cloudflare Tunnel health, restart `cloudflared`.
- **Follow-up**: Open incident ticket; add post-mortem in §24 change log if new class.

G.2 `GitpxWorkerHeartbeatMissing`

- **Trigger**: `time() - max by (worker_id) (gitpx_worker_heartbeat_timestamp_seconds) > 60`.
- **Impact**: Job starvation; queue depth grows.
- **Diagnose**: `systemctl status gitpx-worker@*`, check OOMKilled via journald.
- **Mitigate**: Restart worker; scale out by 1; investigate memory leak via Pyroscope.

G.3 `GitpxRedisStreamBackpressure`

- **Trigger**: `gitpx_queue_depth > 1000` for 15 min.
- **Impact**: Latency degradation.
- **Diagnose**: Identify slow upstream; check `gitpx_push_duration_seconds`.
- **Mitigate**: Temporarily pause that upstream (`gitpxctl pause <upstream>`); drain DLQ; scale workers.

G.4 `GitpxLLMCostBudgetExceeded`

- **Trigger**: `increase(gitpx_ai_cost_usd_total[1d]) > \$BUDGET`.
- **Impact**: Unexpected cloud LLM bill.
- **Diagnose**: Check which role (agent) and model drove cost: `sum by (role, model) (increase(gitpx_ai_cost_usd_total[1d]))`.
- **Mitigate**: Switch LiteLLM router policy for that role to a cheaper model or local Ollama.

G.5 `GitpxForcePushSurge`

- **Trigger**: `rate(gitpx_force_push_total[5m]) > 0.1`.
- **Impact**: Possible misuse or attack.
- **Diagnose**: Review audit log; identify actor and repo.
- **Mitigate**: Revoke actor's token; lock repo via `gitpxctl lock <repo>`.

G.6 `GitpxSignatureVerificationFailure`

- **Trigger**: `rate(gitpx_signature_verified_total{status="fail"}[10m]) > 0`.
- **Impact**: Possible tampering or misconfiguration.
- **Diagnose**: Check which signer (Gitsign/Cosign) and who failed; confirm keys rotated.
- **Mitigate**: Quarantine affected commits/images; require re-sign; investigate signer key status in Rekor.

G.7 `GitpxTunnelDown`

- **Trigger**: `gitpx_tunnel_up == 0` for 2 min.
- **Impact**: Ingress down → no pushes reach proxy.
- **Diagnose**: `cloudflared tunnel info <name>`; check Cloudflare dashboard.
- **Mitigate**: Failover to Tailscale Funnel or secondary tunnel UUID; restart `cloudflared`; if region-wide Cloudflare issue, enable emergency direct-IP DNS record.

G.8 `GitpxSecretDetected`

- **Trigger**: `increase(gitpx_scan_findings_total{scanner=~"gitleaks|trufflehog", severity="high"}[5m]) > 0`.
- **Impact**: Potential leaked credential in push.
- **Diagnose**: Check scan results in Grafana; identify commit + secret type.
- **Mitigate**: **Block the push** (automatic if `secret_policy: block`); rotate credential immediately; rewrite history via BFG + notify owner.

G.9 `GitpxDiskPressureHigh`

- **Trigger**: `node_filesystem_avail_bytes{mountpoint="/"} / node_filesystem_size_bytes < 0.10`.
- **Impact**: Pushes will start failing; Postgres may corrupt.
- **Mitigate**: Run `gitpxctl gc --aggressive`; expand volume; purge Loki retention.

G.10 `GitpxCertExpirationImminent`

- **Trigger**: `probe_ssl_earliest_cert_expiry - time() < 7 * 24 * 3600`.
- **Impact**: Future outage.
- **Mitigate**: Force Caddy / cert-manager renewal; verify ACME challenge path.

---

30. Appendix H — Glossary

Term	Definition
gitpx	The project codename: Git Proxy eXtended.
SSoT	Single Source of Truth. This document.
Smart-Worker	Go daemon that consumes Redis Stream jobs and performs `git push` to upstreams.

Term	Definition
Upstream	A remote Git hosting provider (GitHub, GitLab, …).
Primary	The Gitea instance users push *to*.
UpstreamSet	A YAML manifest describing one or more upstreams (global or per-repo).
Universal Git Adapter	Interface abstraction that all provider plugins implement.
WASM Plugin	A WebAssembly module that customizes adapter behavior at runtime.
Fan-out	Replicating one push to N upstreams.
DLQ	Dead-letter queue: jobs that exceeded retry budget.
PEL	Redis Streams "Pending Entries List": delivered-but-unacked messages.
Cosign keyless	Signing without long-lived keys, using OIDC-issued short-lived certificates.
Gitsign	Keyless Git commit signing via Sigstore.
Rekor	Sigstore's transparency log — immutable record of signatures.
SBOM	Software Bill of Materials.
SLSA	Supply-chain Levels for Software Artifacts.
T1 / T2 / T3	Scale tiers (single-node / regional HA / global mesh).
Force-with-lease	Safer `--force` that fails if upstream ref changed since our last observation.
Intent diff	AI-generated natural-language description of what a commit does.
WebRTC presence	Peer-to-peer live indicator of who is viewing/editing a repo.
ngrok	Alternative tunneling tool (not primary; Cloudflare Tunnel is).
Distroless	Container image with only the app + its dependencies; no shell.
Reproducible build	Bit-identical output given the same inputs.
Ephemeral workdir	A clean tmpfs directory used for a single push, then destroyed.

---

31. Appendix I — Source Document Mapping

Shows which parts of this SSoT were derived from which source document, for traceability back to the original research.

SSoT §	Derived from	Notes
§1 Vision	SRC-A §1, SRC-B §1, SRC-C §1	Unified, expanded into 8 mandates
§2 Evolution	SRC-A §2–3, SRC-B §2	Rejected alternatives formalized
§3 Infrastructure	SRC-B §4 Cloudflare Tunnels, SRC-C §3	Combined
§4 Core architecture	SRC-B §3 Redis Streams	Expanded with UpstreamSet schema
§5 Smart-Worker	SRC-A §4 bash hook → reworked into Go	**Replaced** bash with Go per SRC-B
§6 Adapters	SRC-A §5, §7	Expanded to full provider matrix + WASM
§7 Innovations	**NEW**	15 innovations not in any source
§8 Risks	SRC-A R-001..R-008 + SRC-B R-009..R-010	Extended to R-025
§9 Mitigations	SRC-A §8, SRC-B §5	Per-risk structured
§10 LLM	SRC-A §9, SRC-B §6 LiteLLM	+ LangGraph + DSPy + guardrails
§11 Observability	SRC-A §10, SRC-B §7	Canonical `gitpx_*` prefix
§12 Events	SRC-A §11 hooks, SRC-B §8 Redis Streams	NATS + Apprise + bots added
§13 Languages	SRC-A §12 implicit, SRC-B §9	Full polyglot matrix

SSoT §	Derived from	Notes
§14 Bash SDK	**NEW**	Full framework
§15 GPU/CUDA	**NEW**	vLLM, OpenCV, eBPF
§16 Containers/VMs	SRC-A §13 Docker	Expanded with Firecracker, Kata, KVM
§17 Build systems	SRC-A Makefile	Replaced with Nix+Bazel+Turbo
§18 Testing	SRC-A §14, SRC-B §10	Full 20-layer matrix
§19 Clients	SRC-A §15 Android	Full ecosystem including HarmonyOS, Aurora
§20 Phases	SRC-B §11 phases	Fine-grained to task level
§21 Worldwide scale	**NEW**	T1/T2/T3 migration
§22 Change log	—	This appendix
§23–A Configs	SRC-A §16 examples	Production-ready samples
§24–B Code	**NEW**	Skeleton + safe-push + webhook + Firecracker
§25–C Tools	synthesized	Indexed from all sources
§26–D Free tiers	SRC-B §4	Expanded
§27–E Risk register	§8	Tabular form
§28–F Metrics	§11	Queries added
§29–G Runbooks	§11 alerts	Per-alert
§30–H Glossary	synthesized	
§31–I This map	—	

---

32. Closing — Definition of Done for this Specification

This document is considered **complete and ready for engineering hand-off** when every box below can be checked by the receiving team:

- [x] A single, version-controlled Markdown file (this one) supersedes SRC-A, SRC-B, SRC-C.
- [x] Every inconsistency between source documents is explicitly resolved in §22.
- [x] Every subsystem has (a) a purpose, (b) chosen technology, (c) rationale, (d) a back-link to mitigations and metrics.
- [x] Every risk (R-001..R-025) has a mitigation (§9) and an owner (§27).
- [x] Every metric (§11.2, §28) has a PromQL example and an alert (§11.3, §29) where appropriate.
- [x] Every alert has a runbook (§29).
- [x] Every client platform (web, desktop, Android, iOS, HarmonyOS, Aurora) has an app plan (§19).
- [x] A phased, task-sized delivery plan exists (§20) with estimates and parallel streams.
- [x] A scale path from $0 hobby to global mesh is documented (§21).
- [x] Reference configs (§23) and code snippets (§24) are copy-pasteable starting points.
- [x] Full OSS tool inventory (§25) with licenses.
- [x] Source document traceability (§31) for auditors.

Version: v4.0.0 Unified Blueprint **Codename**: gitpx **Status**: Ready for engineering kickoff (Phase 0 T-0.1). **Next action**: Create `github.com/vasic-digital/gitpx` monorepo per §17, execute T-0.1..T-0.11, and begin Phase 1.

— End of Git Proxy Master Specification —